

第 10 章: Introducing Python Statements (Python 语句入门)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位:这一章是全书从"对象和表达式"走向"语句与逻辑"的桥梁。第 4~9 部分讲了 Python 的内置对象(内置对象、数字、字符串、列表、字典、元组、文件等),但从这一章开始,我们要回答:"程序到底是怎么说'去做这件事'的? "。本章建立的是语句的总体语法模型(缩进、冒号、代码块、语句分隔),后续第 11、12 章才会逐一深入每一个具体语句。这是整个 Python 语法大厦的"地基章",含金量极高。

10.1 开篇引言 (Opening)

英文原文

Now that you're familiar with Python's built-in objects, this chapter begins our exploration of its fundamental statement forms. Much like the previous part's approach, we'll begin here with a general introduction to statement syntax and follow up with more details about specific statements in the next few chapters. In

simple terms, statements are the code we write to tell Python what our programs should do. If, as suggested in Chapter 4, programs "do things with stuff," then statements are the way we specify what sort of things a program does. Less informally, Python is a procedural, statement-based language; by combining statements, we specify a procedure that Python performs to satisfy a program's goals.

中文翻译

既然你已经熟悉了 Python 的内置对象，本章将开始探索 Python 的基本语句形式 (statement forms)。与前一部分的思路类似，我们先从语句语法的一般性介绍入手，然后在接下来的几章里再深入具体的语句细节。简而言之，语句 (statement) 就是我们写给 Python、告诉它"程序应该做什么"的代码。正如第 4 章所说，程序是"用东西做事情" (do things with stuff)，那么语句就是我们规定"程序要做哪种事情"的方式。更正式地说，Python 是一门基于语句的过程式语言 (procedural, statement-based language)；通过组合语句，我们规定了一个 Python 为满足程序目标而执行的过程 (procedure)。

深度理解

·核心概念：本章第一次正式定义"语句"这个概念。回顾前面的章节，我们学会了创建数字、字符串、列表、字典、文件等对象，学会了用表达式去操作它们——但那都是"零件"。语句是把你写的表达式组织成"程序逻辑"的骨架：循环、判断、定义函数，全都是语句。

·底层视角：Python 解释器对源码的处理可以简化为"分词→语法分析→编译为字节码 (bytecode) →解释执行"。语句就是语法分析阶段最基本的单元。if、while、for 这些关键字在词法阶段就被识别为 reserved word (保留字)，参与构成语句结构。

·设计原因：说 Python 是"procedure-based" (过程式)，是因为 Python 的核心执行模型是顺序执行一系列语句。即使你后面学到类、函数，最底层仍然是"一行一行地执行语句"。

·生活例子：把程序想象成一份做菜食谱。食材是"对象"，"把鸡蛋打到碗里"这类动作是"表达式"的用途，而食谱的先后顺序、什么条件下放盐、什么时候出锅——这些"流程控制"就是语句的功能。

·初学者误区：很多初学者把"语句"和"表达式 (expression)"混为一谈。区分方法：表达式有值 (比如 `2 + 3`、`'abc'.upper()`)，语句没有值 (比如 `if x > 0:`)。 `print(x)` 是"表达式语句" (expression statement) ——先用 `print` 函数求值，再丢弃返回值。

10.2 重访 Python 概念层级 (The Python Conceptual Hierarchy Revisited)

英文原文

Another way to understand the role of statements is to revisit the concept hierarchy introduced in Chapte

r 4, which talked about built-in objects and the expressions used to manipulate them. This chapter climbs the hierarchy to the next level of Python program structure: 1. Programs are composed of modules. 2. Modules contain statements. 3. Statements contain expressions. 4. Expressions create and process objects. At their base, programs written in the Python language are composed of statements and expressions. Expressions process objects and are embedded in statements. Statements code the larger logic of a program's operation—they use and direct expressions to process the objects we studied in the preceding chapters. Moreover, statements are where objects spring into existence (e.g., in expressions within assignment statements), and some statements create entirely new kinds of objects (functions, classes, and so on). At the top, statements always exist in modules, which themselves are managed with statements.

中文翻译

理解语句角色的另一个方法是重访第 4 章介绍过的概念层级——那章讲了内置对象以及用来操作它们的表达式。本章将沿层级向上攀登，来到 Python 程序结构的下一层：1. 程序（program）由模块（module）组成。2. 模块包含语句（statement）。3. 语句包含表达式（expression）。4. 表达式创建并处理对象（object）。归根结底，用 Python 语言编写的程序由语句和表达式构成。表达式处理对象，并被嵌入到语句之中；语句则编写了程序运行的更大逻辑。

辑——它们使用并指挥表达式，去处理前面几章学过那些对象。而且，对象正是在语句中"诞生"的（例如，赋值语句中的表达式会创建对象），有些语句还会创造全新种类的对象（函数、类等）。在层级顶端，语句总是存在于模块之中，而模块本身也是靠语句来管理的（import、def、class 这些都是语句）。

深度理解

·核心概念：这是 Python 整个语言模型的"四层金字塔"：对象 表达式 语句 模块。前面几章我们专注于底层两层的"零件"，本章开始向上研究"装配规则"。

·底层实现：当解释器执行一个 .py 文件时，会先把它编译成模块对象（ModuleType），模块的命名空间（namespace）里就是一个个名字→对象的映射，而这些映射正是由语句逐条建立的。dir() 能看到模块里有哪些名字，其实就是"这个模块里跑了哪些语句"的结果。

·设计原因：把层级讲清楚，是让你建立"任何 Python 程序都长这样"的全局观。不管多复杂的程序（Django 网站、机器学习框架），从语法结构上都可以分解到这个四层金字塔。

·生活例子：像盖房子——对象是"砖块水泥"，表达式是"砌墙的动作"，语句是"设计图和施工流程"，模块是"一层楼或一栋楼"，程序是"整个小区"。

·初学者误区：新手常以为"表达式和语句是两种完全不同的代码"。实际上语句是大袋子，表达式是袋子里的东西。比如 `x = [1, 2, 3]` 是一个赋值语句，里面嵌着构建列表的

表达式 [1, 2, 3]。

10.3 Python 的语句 (Python's Statements)

英文原文

Table 10-1 summarizes Python's statement set. Each statement in Python has its own specific purpose and its own specific syntax—the rules that define its structure—though, as you'll see, many share common syntax patterns, and some statements' roles overlap. Table 10-1 also gives examples of each statement, when coded according to its syntax rules. In your programs, these units of code can perform actions, repeat tasks, make choices, build larger program structures, and so on. This part of the book deals with entries in the table from the top through `break` and `continue`. You've informally been introduced to a few of the statements in Table 10-1 already; this part of the book will fill in details that were skipped earlier, introduce the rest of Python's procedural statement set, and cover the overall syntax model. Statements lower in Table 10-1 that have to do with larger program units—functions, classes, modules, and exceptions—lead to larger programming ideas, so they will each have a section of their own. More focused statements (like `del`, which deletes various components) are covered elsewhere in

this book, as well as in Python's standard manuals.

中文翻译

表 10-1 汇总了 Python 的语句全集。Python 中的每条语句都有自己的特定用途，也有自己特定的语法 (syntax)——即定义其结构的规则——不过你会看到，许多语句共享通用的语法模式，有些语句的职责还会重叠。表 10-1 还给出了每条语句按语法规则书写时的示例。在你的程序中，这些代码单元可以执行动作、重复任务、做出选择、搭建更大的程序结构等等。本书本部分处理表中从顶部一直到 `break` 和 `continue` 的条目。你对表 10-1 中的几条语句已经有过非正式的接触；本书本部分将补上前文跳过的细节，介绍 Python 其余的过程式语句，并讲解整体的语法模型。表中位置靠下、涉及更大程序单元（函数、类、模块、异常）的语句，会引出更大的编程主题，所以它们各自会有独立的一节。更聚焦的小语句（比如用于删除各种组件的 `del`）会在这本书的其他地方以及 Python 的标准手册中讲解。

表 10-1: Python 的语句 (Table 10-1. Python statements)

语句 (Statement)	作用 (Role)	示例 (Example)
赋值 (Assignment)	创建引用 (Creating references)	<code>a, b = 'python', 3.12</code> <code>a = (b := next()) + more</code>
调用和其他表达式 (Calls and other expressions)	运行函数 (Running functions)	<code>log.write('app crash')</code>
<code>print</code>	打印对象 (Printing objects)	<code>print(hack, code, file=log)</code>
<code>if/elif/else</code>	选择动作 (Selecting actions)	<code>if 'python' in text: read(text)</code>

match/case	多路选择 (Multiway selections)	match edition: case 6: print(2024)
for/else	迭代 (Iteration)	for x in myiterable: print(x)
while/else	通用循环 (General loops)	while x := file.readline(): print(x)
pass	空占位 (Empty placeholder)	if 'python' not in text: pass
break	退出循环 (Loop exit)	while True: if exits t(): break
continue	循环继续 (Loop continue)	while True: if skiptes t(): continue
def	函数和方法 (Functions and methods)	def f(a, b, c=2, more): print(a + b + c)
return	函数结果 (Functions results)	def f(a, b, c=2, more): return a + b + c
yield	生成器函数 (Generator functions)	def gen(n): for i in n: yield i2
global	命名空间 (Namespaces)	x = 1 + \ndef function(): + \nglobal x; x =
nonlocal	命名空间 (Namespaces)	def outer(): x = 1 + \ndef inner(): nonlocal x; x = 2
async	协程修饰符 (Coroutine designator)	async def consumer(a, b): await producer
await	协程转交 (Coroutine transfer)	await asyncio.sleep(1)
import	模块访问 (Module access)	import sys
from	属性访问 (Attribute access)	from sys import stdin as f
class	构建对象 (Building objects)	class Subclass(Superclass): classAttr = [] def method(self): pa
try/except/finally	捕获异常、收尾动作 (Catching exceptions, termination actions)	try: action() except: print('action error')
raise	触发异常 (Triggering exceptions)	raise EndSearch(location)
assert	调试检查 (Debugging checks)	assert X > Y, 'X too small'

with	上下文管理器 (Context managers)	with open('data') as file: process(file)
del	删除引用 (Deleting references)	del data[k]、del data[i:j]、del obj.attr、del variable
type	类型别名 (Type hinting alias)	type vector = list[float]

英文原文 (关于表 10-1 的若干说明)

Technically, Table 10-1 is sufficient as a quick preview and reference, but it's not quite complete as is. Here are a few fine points about its content:- Assignment statements come in a wide variety of syntax flavors, described in Chapter 11: basic, sequence, augmented, and more; and named assignment (`:=`) is used as an expression, not a statement.- `print` is really a built-in function call, and neither a reserved word nor a statement. Because it will nearly always be run as an expression statement, though, and usually on a line by itself, it's generally thought of as a statement type, and will be treated separately in Chapter 11.- `yield` and `await` are also expressions instead of statements. Like `print`, they're often used as expression statements and so are included in this table, but scripts may also assign or otherwise use their result, as you'll see in Chapter 20. As expressions, `yield` and `await` are also reserved words, unlike `print`.- Most of the words used in statements and expressions are reserved and cannot be used as variables in your code; this includes `and`, `in`, `if`, `for`, `while`, and others. Newer statements use "soft"

reserved words that are reserved only when used in the statements to which they belong; this includes match, case, and type (though not async and await). We'll formalize the full list of reserved words in Chapter 11.

中文翻译 (关于表 10-1 的若干说明)

严格来说, 表 10-1 作为快速预览和参考是够用的, 但它并不完全精准。下面是对其内容的几点补充说明: - 赋值语句有各种各样的语法形态, 第 11 章会介绍: 基本赋值、序列赋值、增强赋值等等; 而命名赋值 (named assignment, 即 `:=`) 是当作表达式使用的, 不是一条语句。- `print` 实际上是一次内置函数 (built-in function) 调用, 既不是保留字也不是语句。但由于它几乎总是以表达式语句的形式运行、而且通常单独占一行, 所以一般把它当作一种语句类型来对待, 会在第 11 章单独处理。- `yield` 和 `await` 同样是表达式而不是语句。和 `print` 一样, 它们常常作为表达式语句使用, 因此被列进了表里; 但脚本也可以把它们的结果赋值给变量或以其他方式使用, 这在第 20 章会看到。作为表达式, `yield` 和 `await` 也是保留字——这一点和 `print` 不同 (`print` 不是保留字)。- 语句和表达式中用到的大部分词汇都是保留字 (reserved word), 不能在你的代码里用作变量名, 这包括 `and`、`in`、`if`、`for`、`while` 等等。较新的语句使用“软”保留字 (soft reserved words) ——它们只有在用于其归属语句时才被保留; 这包括 `match`、`case` 和 `type` (不过 `async` 和 `await` 不是软保留字)。我们会在第 11 章正式列出完整的保留字清单。

深度理解

·核心概念：表 10-1 是 Python 28 种语句的一张"全家福"。注意作者要你抓住的泛化点：很多语句共享同一套语法骨架——if、while、for、try、def、class 全都是"头部行 + 冒号 + 缩进块"的复合语句（compound statement）模式。

·底层实现：这些语句最终都会被编译为不同的字节码指令。例如 if 编译为 POPJUMPIFFALSE/TRUE 加分支跳转；while 编译为条件跳转加回跳；def 会在函数定义的时刻构造一个函数对象（code object + 闭包信息）；class 则会构造一个类对象。所谓"语句"，底层就是一组这样的字节码。

·设计原因：把"print 其实不是语句"这种细节讲清楚，反映了本书一贯的严谨态度——语言事实 > 使用习惯。很多老教程把 print 当作语句讲（Python 2 时代它确实是关键字），但在 Python 3 里它只是内置函数，只是长得像语句而已。

·生活例子：表 10-1 就像一张"汉字部首表"。记下每个部首（语句），再记住"上下结构、左右结构"这些通用书写规则（复合语句的通用语法），你就能组合出任意复杂的字（程序）。

·初学者误区：

·误以为 print 是保留字 → 实际上 print = 3 在 Python 3 里是合法的（虽然会覆盖内置函数，强烈不建议）。

·误以为 `match`、`case`、`type` 是完全保留字 → 它们是"软保留字"，在别的语境下（如 `type` 作为函数、`case` 作为变量名）仍然可用。这也是 3.10+ 版本引入新语句时为了向后兼容而做的权衡。

·误以为 `yield`、`await` 只能在 `def` 里用 → 它们本质是表达式，有结果、可以被赋值。

10.4 两种 if 的故事 (A Tale of Two ifs)

英文原文

Before we delve into the details of any of the concrete statements in Table 10-1, this book wishes to begin our look at Python statement syntax by showing you what you are not going to type in Python code. Consider the following if statement, coded in a C-like language: `c if (x > y) { x = 1; y = 2; }` This might be a statement in C, C++, Java, JavaScript, or similar. Now, look at the equivalent statement in the Python language: `python if x > y: x = 1 y = 2` The first thing that may pop out at you is that the equivalent Python statement is less cluttered—that is, there are fewer syntactic components. This is by design; as a scripting language, one of Python's goals is to make programmers' lives easier by requiring less typing. And less typing also means less room for mistakes. More specifically, when you compare the two syntax models, you'll notice that Python adds one new thing to the mix but remov

es three things that are required in C-like languages.

中文翻译

在深入钻研表 10-1 中任何一条具体语句之前，本书想先从一个角度开始我们对 Python 语句语法的研究：让你看到在 Python 代码里，不会去敲那些东西。考虑下面这条用类 C (C-like) 语言书写的 if 语句：`c if (x > y) { x = 1; y = 2; }` 这可能是 C、C++、Java、JavaScript 或类似语言中的一条语句。现在，看看与之等价的 Python 语句：`python if x > y: x = 1 y = 2` 你第一眼注意到的可能是：等价的 Python 语句更加干净利落——也就是语法组件更少。这是有意为之的；作为一种脚本语言，Python 的目标之一就是让程序员少敲键盘、过得更轻松。少敲键盘也就意味着更少出错的空间。更具体地说，当你对比这两种语法模型时，会发现：Python 往这套“配方”里新增了一样东西，却砍掉了三样在类 C 语言里必不可少的东西。

深度理解

·核心概念：本小节是全章的“语法态度宣言”，也是全书最著名的一处对比教学——用同一个 if 在 C 和 Python 里的样子，直观展示两套语法哲学。这一节引入了全文的“少即是多”价值观。

·底层实现：从词法角度，C 的括号和分号是“显式定界符”；Python 则把“换行 (NEWLINE token)”和“缩进 (INDENT/DEDENT token)”当作定界符。你少敲的括号分号，解释器在词法分析 (tokenizer) 阶段用换行和缩进自动替你补齐了。这也是为什么 Python 没有那种“忘了打

分号"的编译错误，却有 IndentationError。

·设计原因：Guido van Rossum 的设计目标是"让代码符合人的直觉"。人眼天然是按"缩进层级"阅读代码的，Python 索性把缩进变成语法的一部分，让代码看起来的样子就是逻辑真实的样子。

·生活例子：像写信 vs 发短信。C 是写信，要写完整称呼、落款、日期（括号、分号、大括号）；Python 是短信，说完话按回车就完事——"换行=句号"。

·初学者误区：有人觉得"少打符号=功能弱"。恰恰相反，少的只是噪音符号，逻辑能力一点没少——{} 和；在 C 里是给编译器看的机械标记，Python 用缩进把它们变得人类友好。

代码分析

```
// C
if (x > y) {
    x = 1;
    y = 2;
}
```

```
# Python
if x > y:
    x = 1
    y = 2
```

解释器如何处理：Python 解释器读入上面这段代码后，词法分析器（tokenizer）会先看每一行的行首空白：第一行 if 从第一列开始，产生 NEWLINE；第二行 x = 1 前面多了缩进，产生一个 INDENT 记号；执行完 y = 2 后遇到文件结尾，产生 DEDENT（还原缩进层级）。语法分

析器再根据 INDENT/DEDENT 记号，把 $x = 1$ 、 $y = 2$ 归入 if 的 body。最后编译成大致如下的字节码：比较 x 与 y ，若为假则跳过后两条赋值（POPJUMPIFFALSE），为真则依次执行两条 STOREFAST。

设计思想：用换行和缩进替代显式定界符，把"代码之形"与"逻辑之构"合一——这就是 Python "fit your brain" 哲学的语法级体现。

10.5 Python 增加了什么 (What Python Adds)

英文原文

The one new syntax component in Python is the colon character (:). All Python compound statements—statements that have other statements nested inside them—follow the same general pattern of a header line terminated in a colon, followed by a nested block of code usually indented underneath the header line, like this:

Header line: Nested statement block

The colon is required, and omitting it is probably the most common coding mistake among new Python programmers (it's certainly one witnessed thousands of times live in Python training classes). In fact, if you are new to Python, you'll almost certainly forget the col

on character very soon, if you haven't already. You'll get an error message if you do, and most Python-friendly editors make this mistake easy to spot:

```
>>> if x if x ^ SyntaxError: expected':'
```

Including the colon eventually becomes an unconscious habit—so much so that you may start typing colons in your C-like language code, too (generating realms of entertaining error messages from that language's compiler!).

中文翻译

Python 新增的那一个语法成分是冒号 (colon, :)。所有的 Python 复合语句 (compound statement) ——即内部还嵌套着其他语句的语句——都遵循同一个通用模式：一个以冒号结尾的头部行 (header line)，后面跟着一个通常在头部行下方缩进的嵌套代码块，就像这样：Header line: Nested statement block 冒号是必需的，漏掉冒号很可能是 Python 初学者最常见的编码错误（这在 Python 培训课上已经被现场目睹过成千上万次了）。事实上，如果你初学 Python，你几乎很快就会忘记写冒号——如果你还没忘过的话。漏了冒号你会收到一条错误信息，而且大多数对 Python 友好的编辑器都会让这种错误一目了然：>>> if x if x ^ SyntaxError: expected':'

写冒号最终会变成一种无意识的习惯——甚至习惯到你会开始在自己的类 C 语言代码里也敲冒号（然后收到那门语言的编译器给你回敬的一大堆搞笑报错！）。

深度理解

- 核心概念：冒号 = "区块声明开始"的信号。它是复合语句头部行与代码块之间的强制分隔符，也是 Python 语法里唯一一个"你几乎肯定会忘、忘必报错"的成分。
 - 底层实现：在语法层面，冒号让语法分析器知道"头部行到此为止，接下来是子代码块"。它在 tokenizer 阶段被单独识别为 COLON 记号。为什么一定要冒号？因为如果没有它，像 `if x > y:\n a = 1` 这种连续多行的情况，语法分析器将无法确定哪一行是头部、哪一行是子块。
 - 设计原因：冒号同时承担"人类友好的提示"和"机器清晰的定界"两个职责——这也是 PEP 8 和语言哲学中"显式优于隐式"的体现。
 - 生活例子：像中文里的"："引出对话内容——老师：告诉我们下面要开始说话了。Python 的冒号就是在说："条件完了，下面这段缩进的代码属于我。"
 - 初学者误区：
 - 忘了冒号（最常见，作者亲自在培训班见过上千次）。
 - 在单一语句（simple statement，如 `x = 1`）后面也乱加冒号——这同样报错，冒号只属于复合语句的头部。
 - 分不清"函数定义要冒号、列表字面量千万不能加冒号"——`[1, 2, 3]` 中间是逗号不是冒号。
-

10.6 Python 移除了什么 (What Python Removes)

英文原文

Although Python requires the extra colon character, there are three things programmers in C-like languages must include that you don't generally have to code in Python.

中文翻译

虽然 Python 要求多写一个冒号，但在类 C 语言中必须写的三样东西，你在 Python 里 generally（一般而言）都不用写。

10.6.1 括号是可选的 (Parentheses are optional)

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The first of these is the set of parentheses around the tests at the top of some statements: `c if (x < y)` The parentheses here are required by the syntax of many C-like languages. In Python, though, they are not—we simply omit the parentheses, and the statement works the same way: `python if x < y` Technically speaking, because every expression can be enclosed in parentheses, including them will not hurt in this Python code, and they are not treated as an error if present. But don't do that. You'll be wearing out your keyboard needlessly, and broadcasting to the world that you're a programmer of a C-like language still learning Python (it happens). The "Python way" is to simply omit the parentheses in these kinds of statements altogether.

中文翻译

第一样东西是某些语句头部测试条件外围的括号（parentheses）：`c if (x < y)` 在许多类 C 语言中，这里的括号是语法强制要求的。但在 Python 中不是——我们直接省略括号，语句照样工作：`python if x < y` 严格来说，因为任何表达式都可以放进括号里，所以在 Python 代码里写上括号也没有坏处，不会被视作错误。但别这么干。那会白白磨损你的键盘，还会向全世界广播：“我是个还在学 Python 的

类 C 语言程序员" (这种事时有发生) 。"Python 之道" (The Python way) 就是在这类语句中干脆彻底省略括号。

深度理解

- 核心概念: if、while、for 等语句头部后跟的是"表达式", 括号对 Python 语法是多余的。你省下的不只是两个字符, 而是"读屏负担"。
- 底层实现: 语法分析器对 if 后要求的是一个完整的表达式。而括号在 Python 表达式语法中只是"分组设备" (grouping), 不改变优先级语义。所以 `if (x < y):` 与 `if x < y:` 编译出的字节码完全相同。
- 设计原因: C 里括号强制是因为它的语法要求"测试必须用括号包裹"; Python 重新设计了语法, 让头部行的结构更加扁平、易读。这也是"KIS (Keep It Simple)"哲学的胜利。
- 生活例子: 像"确认收货"按钮。C 语言要求你写" (确认: 是)", 命令和参数之间要套一层壳; Python 直接让你写"确认", 干净利落。
- 初学者误区:
 - 从 C/Java 转来的人不自觉写 `if (x > 0):` ——能跑但不够 Pythonic。编辑器 linter 会提示多余的括号 (E731/PEP 8 相关) 。
 - 分不清"表达式求值的括号"与"头部测试的括号"——`x = (a + b) c` 里括号是必需的 (改变优先级), `if (a + b)` 里括号是多余的。

10.6.2 行末即语句结束 (End-of-line is end of statement)

英文原文

The second and more significant syntax component you won't find in Python code is the semicolon. You don't need to terminate statements with semicolons in Python the way you do in C-like languages: `c x = 1; l` In Python, the general rule is that the end of a line automatically terminates the statement that appears on that line. In other words, you can leave off the semicolons, and it works the same way: `python x = 1` There are some ways to work around this rule, as you'll see in a moment (for instance, wrapping code in a bracketed structure allows it to span lines). But, in general, you write one statement per line for the vast majority of Python code, and no semicolon is required. Here, too, if you are pining for your C programming days (and why would you?) you can continue to use semicolons at the end of each statement—the Python language lets you get away with them if they are present, because the semicolon is also a separator when statements are combined. But don't do that either (really!). Again, doing so tells the world that you're a programmer of a C-like language who still hasn't quite made the switch to Python coding. The Pythonic style is to leave off the semicolons altogether. Judging from s

tudents in classes, this seems a tough habit for some veteran programmers to break. But you'll get there; semicolons are useless noise in this role in Python.

中文翻译

从 Python 代码里找不到的第二个、也是更重要的语法成分是分号 (semicolon)。在 Python 里，你不需要像类 C 语言那样用分号来结束语句：`c x = 1`；Python 的通用规则是：一行的结束自动终止该行上的那条语句。换句话说，你可以不写分号，效果完全一样：`python x = 1` 有一些绕开这条规则的办法，你马上就会看到（例如，把代码包进括号结构里，语句就可以跨越多行）。但总的来说，Python 代码的绝大多数情况下是一条语句一行，不需要分号。这里也一样：如果你怀念自己的 C 编程岁月（何必呢？），你当然可以继续每条语句末尾写分号——Python 允许你这么 做而不会报错，因为分号在语句组合时本来就是当分隔符用的。但真的也别这么做！再说一次，这么做同样是在向世界宣告：你是个还没彻底切换到 Python 编程的类 C 语言程序员。Pythonic 的风格是彻底放弃分号。根据课堂上的观察，有些资深程序员确实很难改掉这个习惯。但你一定能改过来；在 Python 里，这个位置上的分号只是无用的噪音。

深度理解

- 核心概念：换行是 Python 语句的天然终止符。你不再需要手动告诉编译器"这句话完了"。
- 底层实现：Python 的词法分析器在处理一行时，如果该行内容构成完整语句，就会发出一个 NEWLINE 记号，语法分析器据此宣告"一条语句结束"。注意它还有隐式拼接

(implicit line joining) 机制：当括号 ()[]{} 尚未闭合时，跨行内容不产生 NEWLINE，这也是"括号换行"技巧（下节会讲）的底层原理。

·设计原因：去掉分号是"少即是多"的典型操作——现实中几乎没有程序员喜欢打分号，它纯粹是编译器的历史包袱（最早因为纸带/行长度限制）。Python 生于 1991 年，敢于彻底抛弃这个包袱。

·生活例子：像现代短信 vs 旧式电报。电报每条以"OVER"结尾（分号），短信按一下发送键（回车）就完事了。

·初学者误区：很多 C/Java/C#/JS 转来的程序员，半年内手会不自觉打字打；。规则明确：单行内多条语句才用分号分隔，行尾的分号纯属无用噪音。现代 IDE 的格式化工具（black、autopep8）会自动帮你把行尾分号去掉。

10.6.3 缩进的结束即代码块的结束 (End of indentation is end of block)

英文原文

The third and final syntax component that Python removes...and the one that may seem the most unusual to soon-to-be-ex-programmers of C-like languages, until they've used it for 10 minutes and realize it's actually a feature...is that you do not type anything explicit in your code to syntactically mark the beginning and end of a nested block of code. You don't need to include begin/end, then/endif, or {/} around the nested block, as you do in C-like languages: `c if (x > y) { x`

`= 1; y = 2; }` Instead, in Python, we consistently indent all the statements in a given single nested block the same distance to the right, and Python uses the statements' physical indentation to determine where the block starts and stops: python `if x > y: x = 1 y = 2` This indentation means the blank whitespace all the way to the left of the two nested statements here. Python doesn't care how you indent (you may use either spaces or tabs), or how much you indent (you may use any number of spaces or tabs). In fact, the indentation of one nested block can be totally different from that of another. The syntax rule is only that for a given single nested block, all of its statements must be indented the same distance to the right. If this is not the case, you will get a syntax error, and your code will not run until you repair its indentation to be consistent:

```
>>> if True: ... a = 1 ... b = 2 b = 2 ^ IndentationError
: unindent does not match any outer indentation level
|
```

中文翻译

Python 移除的第三个、也是最后一个语法成分.....对于那些即将不再是类 C 语言程序员的人来说, 它可能看起来最不寻常——直到你用了 10 分钟, 然后意识到这其实是个福音.....它就是: 你不需要在代码里敲任何显式的东西来语法标记一段嵌套代码块的开始与结束。你不需要像类 C 语言那样, 在嵌套块前后写 `begin/end`、`then/endif` 或 `{/}`: `c if (x > y) { x = 1; y = 2; }` 相反, 在 Python 中, 我们会把

给定嵌套代码块内的所有语句一致地向右缩进到相同距离，Python 就依靠这些语句的物理缩进来判断块从哪里开始、到哪里结束：python if x > y: x = 1 y = 2 这里说的缩进 (indentation)，指的是上面两条嵌套语句左侧一直到行首的那段空白。Python 不在乎你用什么方式缩进（空格或制表符 Tab 都行），也不在乎你缩进多少（任意数量的空格或 Tab 都可以）。事实上，一个嵌套块的缩进可以和另一个嵌套块完全不同。语法规则只有一个：对于给定的单个嵌套块，块内的所有语句必须缩进到相同的右侧距离。如果不满足这一条，你会得到一个语法错误，代码在不修正缩进、使其保持一致之前是无法运行的：>>> if True: ... a = 1 ... b = 2 b = 2 ^ IndentationError: unindent does not match any outer indentation level

深度理解

·核心概念（本章灵魂）：缩进即块结构 (indentation is block structure)。嵌套代码块的范围，由语句行的左缩进层级决定；同一层级 = 同一块，更右 = 更内层，回到更左 = 块结束。

·底层实现：这是 Python 与类 C 语言最深刻的词法差异。tokenizer 维护一个“缩进层级栈”：

·一行以多于栈顶的空格开始时 → 产生 INDENT 记号，入栈；

·一行缩进少于栈顶时 → 逐级产生 DEDENT 记号，出栈；

·栈空时遇到 DEDENT 或在缩进时忽然减少超出已知层级 → 报 IndentationError: unindent does not match an

y outer indentation level。

现代 CPython 还做了一件事：Tab 会被扩展补齐到 8 列（保持向后兼容的算法），再用比较"列号"完成层级判断。SyntaxError 出现在编译阶段——所以缩进错误在运行前就会被发现。

·设计原因：作者给了最有力的理由——可读性 (readability) 是强制性的。如果代码块不是用缩进而是用 {} 标示，那么任何人都可以写出"看起来混乱但机器能跑"的代码；Python 把"写得整齐"从道德要求上升为语法要求。注释里那句"C-like languages might let coders get away with this, but they really shouldn't"道尽了 Python 的态度。

·生活例子：像书籍目录的层级：一级标题顶格、二级缩进两格、三级再缩两格……只要看缩进就知道内容属于哪个层级。Python 就是强迫你把整本书当成目录那样写。

·初学者误区：

·混用 Tab 和空格是最大雷区——即使它们在屏幕上看起来对齐了，解释器按"列号"计算时就可能不一致，报错且很难肉眼排查（现代编辑器默认都做了"Tab 转空格"）。

·"缩进多少无所谓，只要一致就行"是对的，但同一文件里不要乱换缩进宽度，团队通常约定 4 空格 (PEP 8) 或 2 空格。

·空档缩进必须严格等于空格数/Tab 的组合，不要凭感觉"差不多对齐了"。

代码分析

```
if x > y:  
    x = 1  
    y = 2
```

解释器如何处理：tokenizer 逐行读取。第一行 `if x > y:` 以 `if` 开头，产生 `NEWLINE`；第二行 `x = 1` 行首有 4 个空格，栈顶是 0，产生 `INDENT`（缩进层级变量变为 4）；第三行 `y = 2` 同样是 4 个空格，与栈顶匹配，不产生新记号；文件结束时栈非空，`DEDENT` 一次。语法分析器据此构建语法树：`if` 节点的 `body` 包含两条赋值语句。字节码呈现出“块”的物理形态：先求值 `x`、`y` 并比较（`COMPAREOP >`），`POPJUMPIFFALSE` 跳转到 `if` 结束处，随后是两条 `STOREFAST`，最后结束。

设计思想：让“视觉结构 = 逻辑结构 = 字节码结构”三者完全同构。你看代码时的脑内解析，和解释器做的几乎一样——这是 `WYSIWYG`（所见即所得）理念在语法层的最彻底贯彻。

10.7 为什么采用缩进语法？（Why Indentation Syntax?）

英文原文

The indentation rule may seem unusual at first glance to programmers accustomed to C-like languages, but it is an intentional feature of Python: it's one of th

e main ways that Python almost forces programmers to produce uniform, regular, and readable code. It essentially means that you must line up your code vertically, in columns, according to its logical structure. The net effect is to make your code more consistent and readable—unlike much of the code written in C-like languages. To put that more strongly, aligning your code according to its logical structure is a major part of making it readable, and thus reusable and maintainable, by yourself and others. In fact, even if you never use Python after reading this book, you should get into the habit of aligning your code for readability in any block-structured language. Python underscores the issue by making this a part of its syntax, but it's an important thing to do in any programming language, and it has a huge impact on the usefulness of your code. Your experience may vary, but there's a common phenomenon in large, old C++ programs that have been worked on by many programmers over the years. Almost invariably, each programmer has his or her own style for indenting code. For example, a while loop coded in the C++ language may have begun its tenure like this:

`cpp while (x > 0) {` Before we even get into indentation, there are three or four ways that programmers can arrange the `{}` braces in a C-like language, and organizations often suffer political battles and standards docs to address the options (which seems more than a l

little off-topic for the problem to be solved by programming). Be that as it may, here's the scenario often encountered in C++ code. The first person who worked on the code indented the loop four spaces:

```
cpp while (x > 0) {-----; -----;
```

That person eventually moved on to other projects (or, sadly, management), only to be replaced by someone who liked to indent further to the right:

```
cpp while (x > 0) {-----; -----; -----; -----;
```

That person later moved on to other opportunities (ending their reign of coding terror), and someone else picked up the code who liked to indent less:

```
cpp while (x > 0) {-----; -----; -----; -----; -----; -----;}
```

And so on. Eventually, the block is terminated by a closing brace (}), which of course makes this "block-structured code" (yes, sarcasm). No: in any block-structured language, Python or otherwise, if nested blocks are not indented consistently, they become very difficult for the reader to interpret, change, or reuse, because the code no longer visually reflects its logical meaning. Readability matters, and indentation is a major component of readability. Here is another example that may have burned you in the past if you've done much programming in a C-like language. Consider the following statement in C:

```
c if (x) if (y) statement1; else statement2;
```

Which if does the else here go with? Surprisingly, the else is paired with the nested if statement (if (y)) in C, even though it looks visually as though it is associated with the outer test (if (x)). This is a classic pitfall in the C language, and it can lead to the reader completely misinterpreting the code and changing it incorrectly in ways that might not be uncovered until the Mars rover crashes into a giant rock! This cannot happen in Python—because indentation is significant, the way the code looks is the way it will work. Consider an equivalent Python statement:

```
python if x: if y: statement1
else: statement2
```

In this example, the if that the else lines up with vertically and visually is the one it is associated with logically and behaviorally (the outer if x). In a sense, Python is a WYSIWYG language—what you see is what you get—because the way code looks is the way it runs, regardless of who coded it. If this still isn't enough to underscore the benefits of Python's syntax, here's another anecdote. Some companies that develop systems software in the C language, where consistent indentation is not required, enforce it anyhow. It's not too uncommon for such groups to run automated tools that analyze the indentation used in code, when it is checked into source control systems at the end of the

e day. If the tools notice that you've indented your code inconsistently, you might just receive an automated email about it the next morning—and so might your manager! The point is that even when a language doesn't require it, good programmers know that consistent use of indentation has a huge impact on code readability and quality. The fact that Python promotes this to the level of syntax is seen by most as a feature of the language. Also keep in mind that nearly every programmer-friendly text editor has built-in support for Python's syntax model. In the IDLE GUI, for example, lines of code are automatically indented when you are typing a nested block; pressing the Backspace key backs up one level of indentation, and you can customize how far to the right IDLE indents statements in a nested block. There is no requirement on this: four spaces is very common, but it's up to you to decide how and how much you wish to indent (unless your company has endured politics and docs to standardize this too). Indent further to the right for further nested blocks, and less to close the prior block. As a caution, though, you probably shouldn't mix tabs and spaces in the same block in Python, unless you do so consistently; use tabs or spaces in a given block, but not both (in fact, Python issues an error for inconsistent use of tabs and spaces, as you'll see in Chapter 12). Then again, you probably shouldn't mix tabs or spaces in indentation in any structured language—such

code can cause major readability issues if the next programmer has her or his editor set to display tabs differently than yours. C-like languages might let coders get away with this, but they really shouldn't: the result can be a mangled mess. Regardless of which language you code in, you should be indenting consistently for readability. In fact, if you weren't taught to do this earlier in your career, your teachers did you a disservice. Most programmers—especially those who must read others' code—consider it an asset that Python elevates this to the level of syntax. Moreover, generating tabs instead of braces is no more difficult in practice for tools that must output Python code, and the page breaks that can obscure code nesting in the print versions of books (including this one!) will not be present in the real world of coding. In sum, if you do what you should be doing in a C-like language anyhow, but get rid of the braces, your code will satisfy Python's syntax rules.

中文翻译

缩进规则对于那些习惯了类 C 语言的程序员来说乍看之下可能很不寻常，但它是 Python 的一个蓄意特性：它是 Python 近乎“强迫”程序员写出统一、规整、可读代码的主要手段之一。它的本质意思是：你必须让代码按列垂直对齐——对齐到与逻辑结构对应的位置。最终效果是让你的代码更一致、更可读——这与类 C 语言里大量的代码形成鲜明对比。说得更重一点：按逻辑结构对齐代码，是让代码可读、

从而能被自己和他人复用与维护的关键一环。事实上，即使你读完这本书以后再也不用 Python，你也应该养成在任何块结构语言里为可读性而对齐代码的习惯。Python 把这个要求升格为语法，因而更凸显了它的重要性；但这是任何编程语言里都值得做的事，它对你代码的实用价值影响巨大。各人经历不同，但大而旧的 C++ 程序里有一种普遍现象（这类程序通常经年累月被许多人维护过）：几乎毫无例外，每个程序员都有自己的缩进风格。例如，一条写在 C++ 里的 while 循环，起初可能是这样的：cpp while (x > 0) { 还没进入缩进问题之前，程序员在类 C 语言里摆放 {} 括号就有三四种方式，一些组织甚至为这些选项打政治仗、出标准文档（对编程要解决的问题来说，这似乎颇为跑题）。姑且不提这些，下面是在 C++ 代码里常遇到的场景。第一个接手代码的人把循环体缩进了 4 个空格：cpp while (x > 0) {-----;-----; 那个人后来转到别的项目（或者，更可悲的，去做管理了），接手的下一位喜欢缩得更深：cpp while (x > 0) {-----;-----;-----;-----; 那人后来也转去了别的机会（结束了他们的“编码恐怖统治”），又有人接过来代码，这位喜欢缩得浅一些：cpp while (x > 0) {-----;-----;-----;-----;-----;-----; } 就这样不断轮回。最终，代码块由一个右大括号 (}) 收尾——这当然让它成了标准的“块结构代码”（是的，这里是反讽）。不对：在任何块结构语言里，无论 Python 与否，如果嵌套块缩进不一致，读者就会很难理解、修改或复用这段代码，因为代码在视觉上已不再反映它的逻辑含义。可读性至关重要，而缩进是可读性的重要组成部分。下面还有一例，如果你在类 C 语言里写过程序，很可能被它坑过。考虑 C 语言中的这条语句：c if (x) if (y) statement1; else stateme

nt2; 这里的 else 到底跟哪个 if 配对？出乎意料的是，在 C 里这个 else 与嵌套的 if (y) 配对——尽管从视觉上看它像是属于外层测试 (if (x))。这是 C 语言的经典陷阱，可能导致读者完全误解代码、做错误的修改，而且这种错误直到“火星探测器撞上巨岩”时都没被发现！这在 Python 中不会发生——因为缩进是有意义的，代码看起来是什么样，运行起来就是什么样。考虑等价的 Python 语句：`python if x: if y: statement1 else: statement2` 在这个例子里，else 在垂直方向上视觉对齐的那个 if，就是它在逻辑和行为上真正归属的那个 if（即外层的 if x）。从某种意义上说，Python 是 WYSIWYG（所见即所得）语言——代码看起来如何，它就如何运行，无论谁写的都一样。如果这还不足以强调 Python 语法的好处，这里还有一段轶事。一些用 C 语言开发系统软件的公司——C 并不要求一致的缩进——无论如何都强制执行缩进规范。这类团队在代码每天下班签入源码控制系统时，运行自动工具分析代码里的缩进，并不算罕见。如果工具发现你的缩进不一致，第二天早上你可能会收到一封自动化邮件——你的经理也可能收到！要点是：即使语言不要求，优秀的程序员也知道一致的缩进对代码可读性和质量影响巨大。Python 把这一点提升到语法层面，被大多数人视为这门语言的一个特性。另外请记住：几乎所有对程序员友好的文本编辑器都内置了对 Python 语法模型的支持。例如在 IDLE 图形界面里，输入嵌套代码块时行会自动缩进；按 Backspace（退格）键会退回一级缩进；你也可以自定义 IDLE 在嵌套块里把语句缩进到多靠右。这些都没有硬性规定：四个空格最常见，但具体怎么缩、缩多少由你自己决定（除非你的公司也经历了政治和文档之争来标准化它）。要缩得更深，就往右多缩；要结束外层块，就向左回

缩。不过要提醒一句：在 Python 的同一个代码块里，你最好不要混用 Tab 和空格，除非你始终如一地混用；在给定的块里要么用 Tab、要么用空格，但绝不要两者都用（事实上，Python 会为不一致地混用 Tab 和空格报错，第 12 章你会看到）。话又说回来，在任何结构化语言里你都不该混用 Tab 或空格缩进——如果下一位程序员的编辑把 Tab 显示得和你不一样，这种代码会造成严重的可读性问题。类 C 语言可能允许程序员侥幸如此，但它们真不该允许：结果会是一团糟。不管你在哪种语言里写代码，都应当为可读性而保持一致的缩进。事实上，如果你职业生涯早期没被这样教导过，那你的老师们亏欠了你。大多数程序员——尤其是那些必须阅读他人代码的人——都认为"Python 把缩进升格为语法"是一大优点。此外，对必须输出 Python 代码的工具来说，生成 Tab 比生成大括号在实践上并没有更难；而在书籍印刷版里可能弄乱代码嵌套层级的自动分页（包括本书！），在真实编码世界里根本不会存在。总之：只要你做了在类 C 语言里本来就该做的事，然后去掉大括号，你的代码就自然满足 Python 的语法规则。

深度理解

·核心概念：这一节是全书论证"为什么 Python 用缩进做块"的逻辑最密集的一节。三个论据层层递进：①多人 C++ 项目里缩进风格五花八门造成灾难；②C 语言的"悬挂 else" (dangling else) 陷阱——视觉对齐≠逻辑配对；③工业界即便语言不强制缩进，也会用工具强制检查。

·底层实现 (dangling else 的根源)：C 的语法里 else 的配对规则是"就近匹配最近的未配对 if" (动规式贪心)

，这是文法设计决定的，与排版无关，所以会产生视觉误导。人类读者按排版读代码，机器按文法读代码，两者可能不一致——这正是"不可见的结构错误"的温床。Python 用缩进消灭了这个矛盾：文法直接读取缩进，人类与机器第一次看到的是同一件事。

·设计原因（历史的必然）：缩进在 C 时代只是"好习惯"，Python 把它变成"语法"，相当于把"工程师相互依赖的自律"制度化为"语言的强制契约"。正如书里那句犀利的反讽："这当然让它成了标准的'块结构代码'（是的，这里是反讽）"——只靠大括号并不天然等于好的块结构。

·生活例子：像规章文件的层级编号。C 语言允许你写"1.2.1"但大括号其实可以乱放，读者只能靠编号猜想归属；Python 强迫你"每条细则前面必须有正确的层级缩进"，纸面上看到的就是逻辑上真实归属的。

·初学者误区：

·关于"缩进多少"两派争论不休：事实是同一个块内必须一致，4 空格是 PEP 8 社区事实标准，比 2 空格更醒目、比 8 空格更省屏宽。

·有人以为缩进只是"风格问题、可改可不改"——错了，在 Python 里它直接决定块的归属，改错缩进等于改错逻辑。

·混用 Tab/空格：即使肉眼看着对齐，解释器按列计算时 1 个 Tab 折算为 8 列，极易产生诡异的 TabError。在 Python 3 里，混用 Tab 与空格导致缩进歧义是编译期直接报错的。

·连接现实：这节提到的"公司用自动化工具检查缩进"，今天已经演化为 black、ruff format、pre-commit 这类"格式化即 CI 门槛"的工具链——Python 社区把"整齐"变成了工程契约。

代码分析 (dangling else 陷阱对比)

```
// C else if (y) if (x)
if (x)
    if (y)
        statement1;
else
    statement2;
```

```
# Python else if x —
if x:
    if y:
        statement1
else:
    statement2
```

解释器如何处理：C 编译器按语法规则做"就近配对"，else 强行挂到最近的 if(y) 下，与你看到的对齐无关。而 CPython 的语法分析基于缩进层级：else: 与 if x: 处于同一缩进级别（都是 0 级），因此解析器把它归为 if x 的 else 子句；if y: 缩进一级，它拥有自己的独立块。同样的源码直觉，两种语言给出了相反的语义。

设计思想：把排版的自由收走，换来语义的确定。"代码看起来的样子，就是它运行的样子"——这句 WYSIWYG 宣言，是 Python 所有语法决策里最具识别度的一条。

10.8 几个特殊情况 (A Few Special Cases)

英文原文

As mentioned previously, in Python's syntax model:-
The end of a line terminates the statement on that line (without semicolons).- Nested statements are blocked and associated by their physical indentation (without braces). Those rules cover almost all Python code you'll write or see in practice. However, Python also provides some special-purpose rules that allow for flexibility in both statements and nested statement blocks. They're not required and should be used sparingly, but programmers have found them useful in practice.

中文翻译

如前所述，Python 的语法模型是：- 一行的结束终止该行上的语句（不需要分号）。- 嵌套语句通过物理缩进来划分块并建立关联（不需要大括号）。这两条规则几乎覆盖了你在实践中会写或会看到的所有 Python 代码。不过，Python 还提供了一些特殊用途的规则，为语句和嵌套代码块提供更多灵活性。它们并非必需，应当谨慎使用，但程序员们发现它们在实践中确实有用。

深度理解

·核心概念：接下来这几小节是“逃跑通道”。默认规则（换行终止、缩进成块）覆盖 99% 场景，但 Python 为真实世界留了三个特例：多语句挤一行（分号）、单语句跨多

行（括号/反斜杠）、单行复合语句（头部+块同排）。作者反复强调：“用得很省、别滥用”。

·底层实现：这三个特例全部实现于词法/语法层。分号会把 NEWLINE 提前拆成多条 `simplestmt`；未闭合括号抑制 NEWLINE 发出（implicit line joining）；冒号后直接跟语句则该语句并入头部（`simplestmt` 级联）。至于显式反斜杠（\），tokenizer 会吞掉换行符，把物理行合成逻辑行（explicit line joining）。

·设计原因：特例的存在是为了“不逼死极限情况”——比如你想要 `if x: break` 这种一行流、或把超长表达式换行。但为了可读性，这些特例都限制为“只能容纳简单语句”。

·生活例子：像高速公路的应急车道。平时老老实实排队（一条语句一行），真需要超车（单行多语句）或绕路（长语句换行）时，有规则允许，但没人拿它当日常通勤路线。

·初学者误区：把“能”当成“应该”。`a = 1; b = 2; c = 3` 能跑但不该常态化；`print(1); print(2)` 这类写法会被 linter 和代码风格工具强烈警告。

10.8.1 语句规则的特例 (Statement rule special cases)

英文原文

There are three special rules for statements, two of which have already been introduced and used in this book. First of all, although statements normally appear

one per line, it is possible to squeeze more than one statement onto a single line in Python by separating them with semicolons:

```
python a = 1; b = 2; print(a + b) # Three statements
on one line
```

This is the only place in Python where semicolons are required: as statement separators. This only works, though, if the statements thus combined are not themselves compound statements. In other words, you can chain together only simple statements, like assignments, and calls to print and other functions and methods. Compound statements like if tests and while loops must still appear on lines of their own (otherwise, you could squeeze an entire program onto one line, which probably would not make you very popular among your coworkers!). The other special rule for statements is essentially the inverse: you can make a single statement span across multiple lines. To make this work, you simply have to enclose part of your statement in a bracketed pair—parentheses (), square brackets ([]), or curly braces ({}). Any code enclosed in these constructs can cross multiple lines: your statement doesn't end until Python reaches the line containing the closing part of the pair. For instance, to continue a list literal:

```
python mylist = [1111, # Continuation lines
2222, # Any code in (), [], {}
3333]
```

Because this code is enclosed in a square brackets p

air, Python simply keeps reading on the next line until it encounters the closing bracket. The curly braces surrounding dictionaries (as well as set literals and dictionary and set comprehensions) allow them to span lines this way, too, and parentheses handle tuples, function calls, and expressions. The indentation of the continuation lines does not matter, though common sense dictates that the lines should be aligned somehow for readability. Any `#` comments are ignored as usual in continuation lines too, and nested brackets must all be closed before the continuation-line run ends. Parentheses are the catchall device—because any expression can be wrapped in them, simply inserting a left parenthesis allows you to drop down to the next line and continue your statement:

```
python X = (A + B + C + D)
```

This technique works within compound statements, too, by the way. Anywhere you need to code a large expression, simply wrap it in parentheses to continue it on the next line:

```
python if (A == 1 and B == 2 and C == 3): print('hack' 3)
```

An older rule also allows for continuation lines when the prior line ends in a backslash (`\`):

```
python X = A + B + \ C + D # An error-prone older alternative
```

This alternative technique is somewhat discouraged today because it's difficult to notice and maintain the

backslashes. It's also fairly brittle and error-prone—there can be no spaces or `#` comments after the backslash, and accidentally omitting it can have unexpected effects if the next line is mistaken to be a new statement (in this example, `"C + D"` is a valid statement by itself if it's not indented and would silently be run as such). This rule is also a throwback to the C language, where it is commonly used in `"#define"` macros. While `\` may be occasionally useful, when in Pythonland, do as Pythoners do: use bracketed pairs instead of `\` as a rule.

中文翻译

语句规则有三个特例，其中两个本书此前已经介绍并用到过。第一个：虽然语句正常情况下一行一条，但在 Python 里可以用分号把它们分隔开，把多条语句挤到同一行：`python a = 1; b = 2; print(a + b)` # 一行上的三条语句这是 Python 里唯一强制使用分号的地方：作语句分隔符 (statement separator)。不过，只有当这样组合的语句本身不是复合语句时才行。换句话说，你只能串联简单语句 (simple statement) ——比如赋值、对 `print` 及其他函数和方法的调用。像 `if` 测试、`while` 循环这样的复合语句仍必须独占一行（否则，你可以把整个程序都挤进一行里——那大概不会让你在同事中间很受欢迎！）。另一个语句特例基本上是它的逆操作：你可以让一条单条语句横跨多行。要让它奏效，你只需把你语句的一部分包进一对括号里——圆括号 `()`、方括号 `[]` 或花括号 `{}` 都行。任何包在这些结构里的代码都可以跨越多行：你的语句会一直延续，直到 Python 遇到

包含配对右括号的那一行才结束。例如，延续一个列表字面量：`python mylist = [1111, # 续行2222, # (), [], {} 里的任意代码都可换行3333]` 因为这段代码包在方括号对里，Python 会一直读到下一行，直到它遇到右括号为止。包裹字典的花括号（以及集合字面量、字典和集合推导式）同样允许它们这样跨行；圆括号则负责元组、函数调用和表达式。续行的缩进无关紧要，不过常识建议为了可读性还是把它们对齐一下。续行里的 `#` 注释也照常被忽略；而且嵌套的括号在续行结束前必须全部闭合。圆括号是“万能装置”——因为任何表达式都可以包进圆括号，你只要插入一个左圆括号，就可以扔到下一行继续你的语句：`python X = (A + B + C + D)` 顺便一提，这个技巧在复合语句内部同样有效。任何你需要写一个超长表达式的地方，只需用圆括号包住它就能在下一行继续：`python if (A == 1 and B == 2 and C == 3): print('hack' 3)` 一条更古老的规则也允许续行：当前一行以反斜杠（`\`）结尾时：`python X = A + B + \ C + D # 一种易出错的老式替代方案` 这种替代技巧如今已不太被推荐，因为反斜杠很难察觉、也很难维护。它相当脆弱、易错——反斜杠后面不能有空格或 `#` 注释；而且一旦不小心漏掉它，下一行又会被误认为是新语句，可能产生意想不到的效果（在本例中，如果 `"C + D"` 没缩进，它本身就成立一条合法语句，会被悄悄地当作新语句运行）。这条规则也是 C 语言的遗留做法——在 C 里它常被用在 `"#define"` 宏中。虽然 `\` 偶尔还有点用，但在 Python 的世界里就按 Pythonistas 的行事准则来：一律用括号对，而不是 `\`。

深度理解

·核心概念：三种“横向扩展”手段的完整图谱——分号（多

语句合一、仅限简单语句)、括号对(单语句跨行、隐式续行)、反斜杠(老式显式续行)。可靠性与时代性排序为: 括号 > 反斜杠。

·底层实现 (implicit vs explicit line joining) : 这是 CPython tokenizer 的两个著名机制:

·隐式拼接 (implicit line joining) : 维护一个"开放括号深度"计数器。括号深度 > 0 时, 换行符不产生 NEWLINE 记号, 语句持续。注释和空行都可以出现, 因为它们在逻辑行拼接中天然被忽略。

·显式拼接 (explicit line joining) : 当物理行以 \ 结尾时, tokenizer 直接丢弃该换行符。反斜杠一旦后跟空格或注释就失效, 这正是它脆弱的根源。

·设计原因: 为什么推荐括号弃反斜杠? 因为括号有结构性对称(开闭成对、可检测), 反斜杠是无结构标记(丢了没人提醒你), 编译器检测不到"你忘打反斜杠"这件事, 而括号漏了闭合是必报错的。PEP 8 也明确建议: 续行用括号, \ 仅作最后手段。

·生活例子: 括号续行像"信封里装多页纸"——只要信封(括号)没封口(没闭合), 内容就一直算同一封信; 反斜杠像拿胶水粘纸页——胶水(\)断了, 下一张纸就掉出去算一页新的了。

·初学者误区:

·在反斜杠后加了空格或注释, 行过不去, 报 SyntaxError。

·忘记语句内括号配对, 导致 Python 悄悄地把下一行也并

进来，错误发生在很远的地方。

·认为分号后可以跟 if 等复合语句——不行，`x = 1; if x: pass` 直接语法错误。

代码分析

```
#
a = 1; b = 2; print(a + b)      #

mylist = [1111,                #
          2222,
          3333]

X = (A + B +                   #
     C + D)

X = A + B + \                  #
     C + D
```

解释器如何处理：第一行，tokenizer 在该行内遇到两个分号，语法分析器会把它解析成三个连续的 `simplestmt`，字节码依次生成三条 `STOREFAST` 和一次 `PRINTITEM + PRINTNEWLINE`。中间的例子，`[...` 开启的方括号深度为 1，随后的换行不会被当成语句结束，直到 `3333]` 让深度归零、再遇到换行才结束这条语句——所以跨行的列表字面量被编译成一条 `BUILDLIST`。最后一个例子，`\` 结尾让 tokenizer 丢弃换行，物理的两行被拼成一个逻辑行。

设计思想：Python 把“换行=语句结束”作为默认，但用可检测的括号结构提供续行的逃生通道。对比 C 的 `#define` 宏里同样使用 `\` 的事实，说明 Python 刻意选择了比 C 更安全的替代品。

10.8.2 代码块规则的特例 (Block rule special case)

英文原文

As mentioned previously, statements in a nested block of code are normally associated by being indented the same amount to the right. As one special case here, and another we met in earlier chapters, the body of a compound statement can instead appear on the same line as the header in Python, after the colon: `python if x > y: print(x)` This allows us to code single-line if statements, single-line while and for loops, and so on. Much like `;` separators, though, this will work only if the body of the compound statement itself does not contain any compound statements. That is, only simple statements—assignments, calls to `print` and others, and the like—are allowed after the colon. Larger statements must still appear on lines by themselves. Extra parts of compound statements (such as the `else` part of an `if`, which you'll meet in the next section) must also be on separate lines of their own. Compound statement bodies can also consist of multiple simple statements separated by semicolons, but this tends to be frowned upon. In general, even though it's not always required, if you keep most of your statements on individual lines and indent your nested blocks as a norm, your code will be easier to read and change i

n the future. Moreover, some code profiling and coverage tools may not be able to distinguish between multiple statements squeezed onto a single line, or the header and body of a one-line compound statement. It is almost always to your advantage to keep things simple in Python. You can use the special-case exceptions to write Python code that's hard to read, but it takes a lot of work, and there are probably better ways to spend your time. To see a prime and common exception to one of these rules in action, however (the use of a single-line if statement to break out of a loop), and to introduce more of Python's syntax, let's move on to the next section and write some real code.

中文翻译

如前所述，嵌套代码块里的语句通常靠“向右缩进相同距离”建立关联。作为这里的一个特例（也是前面章节见过的另一个特例），复合语句的主体在 Python 中也可以直接出现在与头部同行、紧跟冒号之后的位置：`python if x > y: print(x)` 这让我们可以写单行 if 语句、单行 while 和 for 循环等等。不过，和 ; 分隔符类似，只有当复合语句的主体本身不包含任何复合语句时它才成立。也就是说，冒号之后只允许简单语句——赋值、对 print 等的调用，诸如此类。更大的语句仍必须独占一行。复合语句的附加部分（比如 if 的 else 部分，下一节你将遇到）也必须各自单独占行。复合语句的主体也可以由多个用分号隔开的简单语句组成，不过这种写法通常不被看好。总的来说，即便不总是强制要求，如果你让大多数语句各占一行、并把嵌套块缩进当作常规，你的

代码将来会更容易阅读与修改。而且，有些代码性能分析和覆盖率（coverage）工具可能无法区分"挤在同一行上的多条语句"，或"一行复合语句的头部与主体"。在 Python 里，保持简单几乎总是对你有利。你确实可以用这些特例写出难以阅读的 Python 代码，但那需要费很大功夫，而你多半有更好的方式来花时间。不过，为了看到一个特例规则在实战中的典型常见用法（用单行 if 语句 break 出循环），也为了引入更多 Python 语法，让我们进入下一节，写点真正的代码。

深度理解

- 核心概念：单行复合语句头部：简单语句只在一个场景下被普遍接受——if ...: break / if ...: continue 单行守卫。作者特意在结尾预告了这个"黄金用例"。
- 底层实现：这个特例在语法层面就是把 ifstmt 的 body 允许写成 simplestmt（而非必须缩进的 suite）。它编译出的字节码与"多行缩进写法"完全等价——只是让代码少一行。解释器不偏爱任何一种写法。
- 设计原因：提供"守卫语句（guard）"的便利，同时用"仅限简单语句"的约束防止滥用。因为 if 套 if 一行写出来比多行更难看懂，规则直接禁止。
- 生活例子：像"单行便签"与"正式信函"。便签（if x: break）用于快捷通信；重要通讯（复杂逻辑）必须用信函格式（多行缩进）。
- 初学者误区：
 - 尝试 if x: if y: pass 嵌套一行 → 错误。

- 尝试 `if x: print(1)\nelse: print(2)`, 把 `else` 接到后一行
→ 错误, `else` 必须与 `if` 同层级独占一行。
- 忽视工具链问题: 单行复合语句会干扰调试器和 `coverage`
e 工具的行级统计, 因为它们把多逻辑放一行, 行号失真。

10.9 快速示例: 交互式循环 (A Quick Example: Interactive Loops)

英文原文

You'll see all these syntax rules in action when we tour Python's specific compound statements in the next few chapters, but they work the same everywhere in the Python language. To get started, let's work through a brief but realistic example that demos the way that statement syntax and nesting come together and introduces a few statements along the way. To work along, either copy and paste this section's examples from `media` into your REPL or run them from the file `interact.py` located in this book's examples package using any of the launch tools we studied in Chapter 3.

中文翻译

你会在接下来几章逐个巡视 Python 的具体复合语句时看到所有这些语法规则的运用, 但它们在 Python 语言的任何地方都遵循同一套机制。作为开端, 让我们动手做一个简短但

真实的示例——它演示语句语法与嵌套是如何结合在一起的，并顺路介绍几条新语句。想跟着做的话，你可以从本书示例包（`emedia`）里把本节的例子复制粘贴进你的 REPL，或使用第 3 章学过的任意启动工具直接运行示例包里的 `interact.py` 文件。

深度理解

·核心概念：从本节开始，作者用一个“控制台交互程序”（经典的读-求值-打印循环，`read/evaluate/print loop`）把前面所有语法规则串成实际代码。这是全书第一次写“真正的多行脚本”。

·底层实现：这段脚本如果放到 IDLE 之外跑，我们的理解核心是：`while` 是一个无限循环，靠 `break` 跳出；每次迭代执行 `input()`（阻塞等待键盘输入，返回字符串）、`if` 判断、`print`。

·设计原因：作者坚持“语法学习必须配真实例子”。交互式 REPL（`read-evaluate-print-loop`）本身是 Python 入门最有体验感的闭环。

·生活例子：像客服热线——它不停地问“请问你需要什么帮助”（`input` 提示），你回答后它给反馈（`print` 大写），直到你说“转人工/停止”（`stop` → `break`），热线才挂断。

·初学者误区：`input()` 永远返回字符串（这在后续“数学运算”小节带来一个经典坑）；`while True` 不会自己结束，忘写 `break` 就是死循环。

10.9.1 一个简单的交互式循环 (A Simple Interactive Loop)

英文原文

Suppose you're asked to write a Python program that interacts with a user in a console window. Maybe you're accepting inputs to send to a database or reading numbers to be used in a calculation. Regardless of the purpose, you need to code a loop that reads one or more inputs from a user typing on a keyboard and prints back a result for each. In other words, you need to write a classic read/evaluate/print loop program, similar to Python's standard REPL. In Python, typical boilerplate code for such an interactive loop might look like this:

```
python while True: reply = input('Enter text:') if reply == 'stop': break print(reply.upper())
```

This code makes use of a few new ideas and some we've already explored:

- The code leverages the Python while loop, Python's most general looping statement. We'll study the while statement in more detail later, but in short, it consists of the word while, followed by an expression that is interpreted as a true or false result, followed by a nested block of code that is repeated while the test at the top is true. The word True here is considered always true, so this loop continues forever, unless somehow stopped.
- The input built-in function we met

earlier in the book (to keep windows open after clicks in Chapter 3 and the Appendix A coverage it referenced) is used here for general console input—it prints its optional argument string as a prompt and returns the user's typed reply as a string. - A single-line if statement that makes use of the special rule for nested blocks also appears here: the body of the if appears on the header line after the colon instead of being indented on a new line underneath it. This would work either way, but as it's coded, we've saved an extra line. - Finally, the Python break statement is used to exit the loop immediately—it simply jumps out of the loop statement altogether, and the program continues after the loop. Without this exit statement, the while would loop forever, as its test is always true. In effect, this combination of statements essentially means "read a line from the user and print it in uppercase until the user enters the word 'stop.'" There are other ways to code such a loop (e.g., see the note ahead), but the form used here is very common in Python code and serves to illustrate syntax basics. Notice that all three lines nested under the while header line are indented the same amount: because they line up vertically in a column this way, they are the block of code that is associated with the while test and repeated. Either a lesser-indented statement or the end of the source file (as here) will suffice to terminate the loop body block. When this code is run, either interactively or as a scri

pt file, here is the sort of interaction we get:

```
Enter text: python PYTHON Enter text: 312 312 Enter  
text: stop
```

中文翻译

假设有人要求你写一个在控制台窗口与用户交互的 Python 程序。也许你是在接受要发送给数据库的输入，或是读取要用于计算的数字。不管目的是什么，你都需要编写一个循环：循环从键盘输入的用户那里读取一个或多个输入，并为每个输入打印回一个结果。换句话说，你需要编写一个经典的读/求值/打印循环（read/evaluate/print loop）程序，类似 Python 的标准 REPL。在 Python 中，这类交互式循环的典型样板代码可能是这样的：`python while True: reply = input('Enter text:') if reply == 'stop': break print(reply.upper())` 这段代码用到了几个新概念，也用到了我们之前探索过的一些东西：- 代码利用了 Python 的 `while` 循环——Python 最通用的循环语句。我们稍后会详细学习 `while` 语句；简而言之，它由 `while` 这个词、一个被解释为真或假结果的表达式，以及一段在顶部测试为真时反复执行的嵌套代码块组成。这里的 `True` 这个词始终被视为真，所以这个循环会永远循环下去，除非以某种方式被停止。- 本书早些时候介绍过的内置函数 `input`（第 3 章里用于点击后保持窗口打开、以及附录 A 引用的相关介绍）在这里用于通用的控制台输入——它把可选参数字符串作为提示符打印出来，并把用户输入的回答作为字符串返回。- 这里也出现了一条利用嵌套块特例规则的单行 `if` 语句：`if` 的主体出现在冒号之后的头部行上，而不是缩进到下面的新行。两种写法

都行，但按现有写法，我们省了一行。- 最后，Python 的 `break` 语句用于立即退出循环——它干脆利落地从整个循环语句里跳出来，程序继续执行循环之后的代码。没有这条退出语句，`while` 就会永远循环下去，因为它的测试条件恒为真。实际上，这组语句的组合本质上是说："从用户那里读一行，把它打印成大写，直到用户输入单词'stop'。"写这样的循环还有其他方式（例如，见下方注释），但这里用的形式在 Python 代码中非常常见，很适合用来演示语法基础。注意，`while` 头部行下方嵌套的三行都缩进了相同的距离：正因为它们这样垂直对齐成一行，才成为与 `while` 测试关联、并被反复执行的代码块。无论是一条缩进更少的语句，还是源文件的结尾（如这里所示），都足以终止循环体代码块。当这段代码运行时——无论以交互方式还是作为脚本文件——我们得到的交互大致是这样的：Enter text: python PYTHON Enter text: 312 312 Enter text: stop

深度理解

·核心概念：整段代码是三条语句的"协作演出"：`while True`（永恒的循环框架）、`if reply == 'stop': break`（单行守卫，退出循环）、`print(reply.upper())`（处理结果）。这就是 READ-EVAL-PRINT LOOP 最朴素的实现。

·底层实现：字节码层面，`while True` 编译为：循环头无条件跳回自己；`break` 编译为 JUMPABSOLUTE 跳出 `while` 的循环体末尾；`reply.upper()` 是一次 LOADATTR + CALL；如果输入'312'，`upper()` 对数字字符串无效但无害（返回原样）。`input()` 在 CPython 里调用 C 库 PyOS Readline，阻塞等待换行。

·设计原因：if ...: break 之所以优雅，是因为它把"退出条件"作为循环体内的一个提前出口，而非把 while 条件写得冗长。这也呼应了上一节"单行守卫是特例黄金用例"的说法。

·生活例子：像餐厅的"点单确认"呼叫铃——服务员（循环）反复问"要点什么？"（input 提示），客人说"埋单"（stop）就停止服务（break）。

·初学者误区：

·input() 返回的是字符串，若输入 312 打印出来还是 312（字符串）而非整数——下一节会引爆这一点。

·忘写 break 的条件分支 → 程序永远无法退出，只能 Ctrl + C。

·把 while True 误写成 while 1——两者几乎等价，但 True 语义清晰（历史上 1 是祖传写法）。

代码分析

```
while True:
    reply = input('Enter text:') #
    if reply == 'stop': break     # stop
    print(reply.upper())         #
```

解释器如何处理：CPython 把源码编译成字节码后，显然是一个"测试-分支-回跳"的三明治结构：while 头部先无条件跳到判断点，判断 True 恒真进入循环体；循环体依次执行 CALLFUNCTION (input)、COMPAREOP ==、POPJUMPIFFALSE（若不等则跳过 break）、break 翻译成的跳出循环末尾的跳转指令。upper() 是字符串方

法的调用，在字节码里表现为 LOADMETHOD + CALLMETHOD，然后 PRINT 系列指令输出。

设计思想：while True + break 是 Python 中"直到型循环"的惯用法（do-until 的替代品）。它把"循环条件"与"退出条件"分离：外层条件固定为真，真正的终止诅咒由循环体内的守卫语句判发——这种结构比"文档式复杂条件"更易读、更常见于真实项目。

10.9.2 用 := 运算符压缩代码 (NOTE: Or crunch code with the := operator)

英文原文

NOTE: Spoiler alert—this section's code is traditional and simple and works as a syntax demo, but it's possible to reduce it from four lines to two with the := named-assignment expression added in Python 3.8 and covered in the next chapter. This expression assigns a name to another expression's result, but also returns the assigned value as its overall result. The net effect lets us fetch, assign, and test input on the same line—and all in the loop's header: `python while (reply := input('Enter text:')) != 'stop': print(reply.upper())` While useful in narrow roles, this expression also requires nested parentheses in this context and is arguably more implicit than traditional forms (though ex-C programmers' mileage may vary!).

中文翻译

注释 (NOTE)：剧透警告——本节的代码传统而简单，作为语法演示很合适；但其实可以用 Python 3.8 加入、将在下一章详细讲解的 `:=` 命名赋值表达式 (named-assignment expression)，把这段四行代码压缩成两行。这个表达式把名字赋给另一个表达式的结果，同时把被赋的值作为整体结果返回。最终效果是让我们能在同一行里完成"获取、赋值、测试"输入——而且全部写进循环的头部：`python while (reply := input('Enter text:')) != 'stop': print(reply.upper())` 虽然在狭窄的场景中有用，但这个表达式在这种上下文里需要嵌套括号，而且可以说比传统写法更隐式（不过那些前 C 语言程序员的感受可能因人而异！）。

深度理解

·核心概念：`:=`（读作 "walrus operator" / 海象运算符）是"赋值同时返回值"的表达式。它让"读输入 → 存变量 → 判断"三步并一步，且值仍然留在变量 `reply` 里供循环体使用。

·底层实现：`while` 头部要求的"一个表达式"在这里是 `(reply := input(...)) != 'stop'`。字节码层面，海象运算编译为"把 RHS 求值结果 STOREFAST 到 `reply`，同时把同一结果留在栈顶"，随后与 `'stop'` 比较。关键：海象必须在括号里，因为裸写 `while reply := ...` 会导致优先级歧义，语法明确要求包括号。

·设计原因：作者用这则 NOTE 展示"传统简洁"与"现代简短"的权衡。海象运算符的争议核心就是"隐式 vs 显式"：

它减少重复（不用再 `reply = ...` 之后 `while reply != ...`），但读起来不够明显。PEP 572 是用"must be parenthesized to be unambiguous"来约束它的。

·生活例子：像"收银找零"一体机——传统方式是"扫码→记到本子上→再算钱"三步；海象是一台机器"扫码的同时就把金额打进账本并弹出找零"。效率高，但你要懂这台机器的逻辑才行。

·初学者误区：

·忘括号：`while reply := input():` 会报语法错误，必须 `(reply := input(...))`。

·混淆赋值 = （语句，无值）与海象 := （表达式，有值）——这正呼应了章节开篇"表达式有值、语句无值"的区分。

。

·在 `def` 参数、`:=` 的绑定目标不能重名等细微限制上出错。

。

10.9.3 对用户输入做数学运算 (Doing Math on User Inputs)

英文原文

Our script works, but now suppose that instead of converting a text string to uppercase, we want to do some math with numeric input—squaring it, for example (perhaps in some misguided effort of an age-input program to tease its users). We might try statements like these to achieve the desired effect:

```
>> reply = '40'>> reply ** 2
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

This won't quite work in our script, though: input from a user is always returned as a string, and as discussed in the prior part of this book, Python won't convert object types in expressions unless they are all numeric. We cannot raise a string of digits to a power unless we convert it manually to an integer:

```
>> int(reply) ** 2
1600
```

Armed with this information, we can now recode our loop to perform the necessary math. Type the following in a file to run it with command line, IDLE menu options, or any other technique we met in Chapter 3 (the REPL both requires a blank line after the while, and runs just one statement at a time—the final print here wouldn't make sense):

```
python while True: reply = input('Enter text:')
    if reply == 'stop': break
    print(int(reply) ** 2)
    print('Bye')
This script uses a single-line if statement to exit on "stop" as before, but it also converts inputs to perform the required math. This version also adds an exit message at the bottom. Because the print statement in the last line is not indented as much as the nested block of code, it is not considered part of the loop body and will run only once, after the loop is exited:
```

```
Enter text: 2
4
Enter text: 40
1600
Enter text: stop
Bye
```

中文翻译

我们的脚本能跑，但现在假设我们不想把文本字符串转成大写，而是想对数值输入做点数学运算——比如求它的平方（或许是某个“输入年龄”程序为了逗弄用户的馊主意）。我们可能会尝试像下面这样的语句来达到目的：

```
>>> reply = '40'>>> reply ** 2
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

但在我们的脚本里这样可不行：用户输入总是以字符串形式返回，而且正如本书前一部分讲过的，除非全都是数字类型，否则 Python 不会在表达式里自动转换对象类型。我们无法用一个数字字符串去求幂，除非手动把它转换成整数：

```
>>> int(reply) ** 2
1600
```

有了这个信息，我们就可以重新编写循环来完成所需的数学运算。把下面代码输入到一个文件，用命令行、IDLE 菜单选项或我们在第 3 章学到的任何其他技术来运行（REPL 要求 while 后要有空行，而且一次只运行一条语句——这里的最后一条 print 在 REPL 里没意义）：

```
python while True:
    reply = input('Enter text:')
    if reply == 'stop':
        break
    print(int(reply) ** 2)
print('Bye')
```

这个脚本和之前一样，用一条单行 if 语句在遇到 "stop" 时退出，但它也把输入转换成数字来做所需的数学运算。这个版本还在底部加了一条退出信息。因为最后一行的 print 语句没有像嵌套代码块那样缩进，它就不被认为是循环体的一部分，只会在循环退出后运行一次：

```
Enter text: 2 4
Enter text: 40 1600
Enter text: stop
Bye
```

深度理解

·核心概念：这是全书的第一堂"类型错误"现场课——`input()` 永远是字符串，Python 不做隐式类型转换（除非全是数值）。`'40' 2` 直接 `TypeError`。解决办法：`int(reply)` 显式转换。同时，"最后一行去掉缩进"这个动作，正式演示了缩进终结块的语法规则。

·底层实现：`'40' 2` 报错源于 Python 的动态类型 + 操作符调度：调用 `str.pow`，字符串类型未实现幂运算，于是再尝试反射操作 `int.rpow`，仍不匹配，最终抛 `TypeError`。而 `int('40')` 走 `int.new` 的字符串解析路径，把十进制字符串转成 C long。字节码里，`print(int(reply) 2)` 会先 `CALLFUNCTION` 构造 `int`，再 `BINARYPOWER`，最后打印。

·设计原因：Python 的"拒绝隐式转换"是对"惊喜最小化"的坚持——如果 `'40' 2` 悄悄算出 1600，那么 `'abc' 2` 和 `'abc' + 2` 呢？语言选择：字符串和数字的混合就让它报错，逼你自己明确表达意图。这比 C 的 `atoi` 隐式转换安全得多。

·生活例子：像"买水果"——收银员不会因为你说"四十"就自动当成"40 斤"，你必须明确说单位。`int()` 就是"把话说成数字"的翻译官。

·初学者误区：期待 `int` 能处理 `'40.5'`（会 `ValueError`）、期待 `int('040')` 出问题（其实能转）、以为 `input` 能"智能"返回数字。记住一条铁律：`input` 的返回值永远是 `str`，需要数字就手动转换。

代码分析

```
while True:
    reply = input('Enter text:')      #
    if reply == 'stop': break         #
    print(int(reply) ** 2)             # int()
print('Bye')                          # while
```

解释器如何处理：前四行的循环体以 4 空格缩进成块；最后一行 `print('Bye')` 缩进为 0，tokenizer 在读到它时产生 `DEDENT`，语法分析器据此判定 `while` 块在这一行结束。因此字节码里 `'Bye'` 的打印在循环末尾之外。同理解释 REPL 的提示："REPL 里 `while` 后必须有空行"是因为交互模式下无法仅凭缩进判断块结束，空行就是"我要退出这个块"的信号。

设计思想：同一个"缩进即块"规则同时完成了"同层成块"与"退回出块"两件事。仅凭一个缩进层级的变化（从 4 变 0），代码就从"循环里的重复行为"变成了"循环后的收尾行为"——这正是为什么"代码的样子 = 逻辑的样子"。

10.9.4 通过测试输入来处理错误 (Handling Errors by Testing Inputs)

英文原文

So far so good, but notice what happens when the input is invalid:

```
Enter text: xxx ValueError: invalid literal for int() with
base 10:'xxx'
```

The built-in `int` function raises an exception (i.e., flag s an error) here in the face of a nonnumber. If we want our script to be robust, we can check the string's content ahead of time with the string object's `isdigit` method:

```
>> S='40'>> T='xxx'>> S.isdigit(), T.isdigit() (True, False)
```

This also gives us an excuse to further nest the statements in our example. The following new version of our interactive script uses a full-blown `if` statement to work around the exception on conversion errors:

```
python while True: reply = input('Enter text:') if reply == 'stop': break
```

```
elif not reply.isdigit(): print('Bad!' 8) else: print(int(reply) 2) print('Bye')
```

We'll study the `if` statement in more detail in Chapter 12, but it's a fairly lightweight tool for coding logic in scripts. In its full form, it consists of the word `if` followed by a test and an associated block of code; one or more optional `elif` ("else if") tests and code blocks; and an optional `else` part with a block of code at the bottom to serve as a default. Python runs the block of code associated with the first test that is true, working from top to bottom, or the `else` part if all tests are false.

The `if`, `elif`, and `else` parts in the preceding example are associated as part of the same statement because their opening words all line up vertically (i.e., share the

e same level of indentation). The if statement spans from the word if to just before the print statement on the last line of the script. In turn, the entire if block is part of the while loop because all of it is indented under the loop's header line. Statement nesting like this is natural once you start using it.

When we run our new script, its code catches errors before they occur and prints an error message before continuing (which you'll probably want to improve before this code is handed over to the quality-assurance team), but "stop" still gets us out, and valid numbers are still squared:

```
Enter text: 5 25 Enter text: xxx Bad!Bad!Bad!Bad!Bad!
Bad!Bad!Bad! Enter text: 10 100 Enter text: stop Bye
```

中文翻译

到目前为止一切顺利，但注意当输入无效时会发生什么：Enter text: xxx ValueError: invalid literal for int() with base 10:'xxx'面对非数字输入，内置函数 int 会抛出一个异常 (exception) (也就是标记一个错误)。如果想让脚本更健壮，我们可以先用字符串对象的 isdigit 方法提前检查字符串的内容：>>> S = '40'>>> T = 'xxx'>>> S.isdigit(), T.isdigit() (True, False) 这同时也给了我们一个把示例继续嵌套下去的理由。下面这个新版本在我们的交互脚本里用了一条完整的 if 语句，来绕过转换错误时的异常：python while True: reply = input('Enter text:') if reply == 'st

op': break elif not reply.isdigit(): print('Bad!' 8) else: print(int(reply) 2) print('Bye') 我们在第 12 章会更详细地学习 if 语句，但它其实是脚本里编写逻辑的相当轻量的工具。在完整形态下，它由 if 这个词加上一个测试及关联的代码块构成；后面可以跟一个或多个可选的 elif ("else if") 测试及其代码块；最后还可以有一个可选的 else 部分，带一个代码块作为默认兜底。Python 从上到下，运行第一个为真的测试所关联的代码块；如果所有测试都为假，则运行 else 部分。上面这个例子里，if、elif、else 各部分之所以被关联为同一条语句，是因为它们的起首词都垂直对齐（即共享同一缩进层级）。这条 if 语句从 if 这个词一直延伸到脚本最后一行 print 语句之前的那个位置。反过来，整个 if 代码块又是 while 循环的一部分，因为它全部缩进在循环头部行之下。一旦用起来，像这样的语句嵌套是很自然的事。当我们运行新脚本时，它的代码在错误发生之前就把错误拦截下来，先打印一条错误信息再继续（这段报错提示在你把它交给质量保证团队之前，大概还是改一改比较好）：但 "stop" 依旧能让我们退出，合法数字也依旧被求平方：Enter text: 5 25 Enter text: xxx Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad! Enter text: 10 100 Enter text: stop Bye

深度理解

·核心概念：本小节集中展示 if / elif / else 三件套和一个极重要的嵌套判定：if、elif、else 同缩进才算同一条语句。同时首次讲述"防御式编程"路径——用 isdigit() 在转换前验证。

·底层实现：字节码层面，if/elif/else 编译成"链式条件跳转 + 兜底标签"的决策树：先测 reply == 'stop'，判定真

走 break; 假则测 not reply.isdigit(), 真走'Bad!' 分支; 再假走 else 分支计算平方。str.isdigit() 是一个 C 实现的方法, 逐字符检查是否全为十进制数字。注意'40.5'.isdigit() 是 False、'4 0'.isdigit() 是 False——它的定义非常严格。

·设计原因: .isdigit() 方案属于"测试驱动、防御式"风格; 下一小节 try/except 则是"先假设成功、出错再处理"风格。作者特意把两种哲学并排展示——这是理解 Python 异常哲学 (EAFP vs LBYL) 的入门点。

·生活例子: if/elif/else 像医院分诊台——先问"是不是急诊" (if), 不是再问"是不是普通号" (elif), 都不是就排"挂号窗口" (else 兜底)。所有环节按优先级一次只走一条。

·初学者误区:

·把 elif 写成 else if——Python 没有 else if, 就是 elif。

·误以为多个 if 和 elif 一样——多个 if 会全部判断, if/elif 命中一个就"短路"跳过后面的, 行为完全不同。

·isdigit 判断负数和浮点数字符串都会失败 ('-3'、'3.14' 都对 isdigit 说不), 这正是作者下一节选择 try 处理浮点的原因。

代码分析

```

while True:
    reply = input('Enter text:')          #
    if reply == 'stop':                    #
        break
    elif not reply.isdigit():              #
        print('Bad!' * 8)
    else:                                  # int
        print(int(reply) ** 2)
print('Bye')

```

解释器如何处理：注意 break、打印'Bad!'、打印平方分别缩进在 if/elif/else 之下（8 空格），而 if/elif/else 三个起首词在同一缩进列（4 空格）。tokenizer 用缩进层级把"elif 属于这条 if"这种关联翻译给语法分析器。字节码表现为：比较 ==，POPJUMPIFFALSE 跳到 elif 测试；elif 测试 not reply.isdigit()，真则打印'Bad!'；false 则进入 else，执行 int + + print。整个决策链被 while 的外层循环包围，循环结束后才执行最后的 print('Bye')。

设计思想：结构即逻辑的可读性红利在此完美展现——只凭缩进，你一眼就能看出"哪里是循环、哪里是分支、哪里是收尾"。嵌套虽是"自然"的，但作者也在暗示：嵌套越深可读性越差，能压平就压平。

10.9.5 用 try 语句处理错误 (Handling Errors with try Statements)

英文原文

The preceding solution works, but as you'll see later in the book, the most general way to handle errors in

Python is to catch and recover from them completely using the Python try statement. We'll explore this statement in depth in Part VII of this book, but as a preview, using a try here can lead to code that some might see as simpler than the prior version:

```
python while True: reply = input('Enter text:') if reply == 'stop': break try: num = int(reply) except: print('Bad!' 8) else: print(num 2) print('Bye')
```

This version works exactly like the previous one, but we've replaced the explicit error check with code that assumes the conversion will work and wraps it in an exception handler for cases when it doesn't. In other words, rather than detecting an error, we simply respond if one occurs. This try statement is another compound statement and follows the same pattern as if and while. It's composed of the word try, followed by the main block of code (the action we are trying to run), followed by an except part that gives the exception handler code and an else part to be run if no exception is raised in the try part. Python first runs the try part, then runs either the except part (if an exception occurs) or the else part (if no exception occurs). In terms of statement nesting, because the words try, except, and else are all indented to the same level, they are all considered part of the same single try statement. Notice that the else part is associated with the try here, not the if. As we've seen, else can appear in if statements in Python, but it can also appear in try statements and loops—its indentation t

ells you what statement it is a part of. In this case, the try statement spans from the word try through the code indented under the word else, because the else is indented the same as try. The if statement in this code is a one-liner and ends after the break, so the else cannot apply to it.

中文翻译

前面的方案能跑，但正如你将在本书后面看到的，Python 里处理错误最通用的方式是使用 try 语句把异常完整地捕获并恢复。我们会在本书第 VII 部分深入探讨这条语句，但这里先预览一下：在这里用 try 写出的代码，有些人可能觉得比前一版本更简洁：

```
python while True: reply = input('Enter text:')
if reply == 'stop': break
try: num = int(reply)
except: print('Bad!' 8)
else: print(num 2)
print('Bye')
```

这个版本和前一版本的行为完全一致，但我们把显式的错误检查替换成了这样的代码：先假设转换会成功，如果没成功，就交给异常处理器兜底。换句话说，我们不去“检测”错误，而是“如果错误发生了，就作出响应”。这条 try 语句是又一个复合语句，遵循与 if、while 相同的模式。它由 try 这个词、主代码块（我们试图运行的动作）、一个提供异常处理器代码的 except 部分，以及一个在 try 部分没有抛出异常时要运行的 else 部分组成。Python 先运行 try 部分，然后运行 except 部分（如果发生了异常）或 else 部分（如果没有发生异常）。从语句嵌套的角度看，因为 try、except、else 三个词缩进到了同一层级，所以它们都被视为同一条 try 语句的一部分。注意，这里的 else 是与 try 关联的，而不是与 if 关联。正如我们所见，else 在 Python

中可以出现在 if 语句里，但也可以出现在 try 语句和循环里——它的缩进告诉你它属于哪条语句。在本例中，try 语句从 try 这个词一直延伸到 else 下面缩进的那些代码处，因为 else 与 try 的缩进一致。而代码里的那条 if 语句是个单行语句，在 break 之后就结束了，所以这个 else 不可能属于它。

深度理解

·核心概念：本小节是全书第一次正式面对 try/except/else，并抛出两种错误处理哲学的对比：LBYL (Look Before You Leap, 先检查再执行，上节的 isdigit 方案) vs EAFP (Easier to Ask Forgiveness than Permission, 先做、错了再求饶，本节的 try 方案)。Python 社区整体更偏好 EAFP。

·底层实现：try 的字节码是"受保护区域" (protected block)：进入 try 块时，记录异常处理表的 entry (起始偏移、异常类型、except 处理句柄的跳转目标)。若 int 抛 ValueError，异常传播机制查到该偏移落入保护区，就跳转到 except 处理代码；若一切正常，跳转到 else 块；两种分支之后做"异常表清理"。except: (不带类型) 会把所有异常一律吞掉——包括 KeyboardInterrupt。

·设计原因：为什么 else 要出现在 try/except/else? 为了精确控制"正常路径"——else 里的代码只在"没异常"时执行。若不写 else 而把代码直接放在 try 里，那么这段代码自己抛的异常也会被同一个 except 捕获，逻辑混淆。这正是 python try-else 的精妙之处。

·生活例子：EAFP 像"先刷卡再响提示音"——你不必先查银行卡余额（先检查），直接刷，没钱就滴滴报警（异常），有钱就顺利通行（else）。LBYL 是先查余额再决定刷不刷。

·初学者误区：

·except: 裸捕获一切，掩盖真实的 KeyboardInterrupt 和编程错误——真实项目应先 except ValueError 之类的具体类型。

·分不清"这个 else 属于 if 还是 try"——答案永远看缩进。

·以为 try 只包"可能出错的那一行"就行——正好相反，else 块的存在就是鼓励你把依赖成功结果的逻辑也放进来，统一管理正常路径。

代码分析

```
while True:
    reply = input('Enter text:')          #
    if reply == 'stop': break              # if break
    try:
        num = int(reply)                  # ValueError
    except:
        print('Bad!' * 8)                  #
    else:
        print(num ** 2)                    # num
print('Bye')
```

解释器如何处理：把 try/except/else 想象成一个受管辖区间。执行到 int(reply) 时若输入非法，CPython 的异常机制（PyErr 状态 + 线程异常表）会循着调用栈回溯，匹

配到本帧里"异常表"登记的 try 区间，转入 except 处理器，执行'Bad!' 打印；之后不会再回头执行 int(reply) 之后的 num = ...，而是跳过整个 try 块。若转换成功，num 被 STOREFAST 绑定，控制流走 else，执行 num 2 后再打印。两种路径最后都回到 while 循环头。if ... break 同前。

设计思想：异常不是"洪水猛兽"，而是 Python 官方的控制流通道。用异常做错误分流，可以省掉大量前置校验代码，让"正常路径"读起来最顺（主路直行，岔路在 except 里收尾）。但它的前提是你明确知道要捕获什么异常，裸 except: 是新手第一道要跨过的坎。

10.9.6 支持浮点数 (Supporting Floating-Point Numbers)

英文原文

Again, we'll come back to the try statement later in this book. For now, be aware that because try can be used to intercept any error, it reduces the amount of error-checking code you have to write, and it's a very general approach to dealing with unusual cases. If we're sure that print won't fail, for instance, this example could be even more concise:

```
python while True: reply = input('Enter text:') if reply == 'stop': break try: print(int(reply) 2) except: print('Bad!' 8) print('Bye') And if we wanted to support input of floating-point numbers instead of just integers, for
```


example, using try would be much easier than manual error testing—we could simply run a float call and catch its exceptions:

```
python while True: reply = input('Enter text:') if reply == 'stop': break try: print(float(reply) / 2) except: print('Bad!') print('Bye')
```

There is no isfloat method for strings today, so this exception-based approach spares us from having to accommodate all possible floating-point syntax in an up-front error check (a nontrivial task, given the many faces of floats!). When coded this way, we can enter a wider variety of numbers, but errors and exits still work as before:

```
Enter text: 50 2500.0 Enter text: 40.5 1640.25 Enter text: 1.23E-100 1.5129e-200 Enter text: hack Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad! Enter text: stop Bye
```

NOTE: Or run anything with eval and exec: Python's built-in eval call, which we used in Chapters 5 and 9 to convert data in strings and files, would work in place of float here, too, and would allow input of arbitrary expressions ("2 100" would be a legal, if curious, input, especially if we're assuming the program is processing ages!). This is a powerful concept that is open to the same security issues mentioned in the prior chapters. If you can't trust the source of a code string, use more focused and restrictive conversion tools like int and float.

Python's `exec`, used in Chapter 3 to run code read from a file, is similar to `eval` (but assumes the string is a statement instead of an expression and has no result), and its compile call precompiles frequently used code strings to bytecode objects for speed. Run a help on any of these for more details. We'll also use `exec` to import modules by name string in Chapter 25—an example of its more dynamic roles.

中文翻译

再说一遍，本书后面还会回到 `try` 语句。现在只要知道：因为 `try` 可以用来拦截任何错误，它减少了你不得不写的错误检查代码，是处理异常情况的一种非常通用的手段。比如，如果我们确信 `print` 不会失败，这个例子甚至可以更简洁：

```
python while True: reply = input('Enter text:') if reply == 'stop': break try: print(int(reply) 2) except: print('Bad!' 8) print('Bye')
```

而如果我们想要的不是只支持整数，而是支持浮点数输入，用 `try` 就会比手工错误测试容易得多——我们只需运行一次 `float` 调用并捕获它的异常：

```
python while True: reply = input('Enter text:') if reply == 'stop': break try: print(float(reply) 2) except: print('Bad!' 8) print('Bye')
```

目前字符串并没有 `isfloat` 这样的方法，所以这种基于异常的方式省去了我们“在前置检查里把所有可能的浮点语法都考虑进来”的麻烦（考虑到浮点数的千变万化，这可不是个小任务！）。这样写之后，我们可以输入更多种类的数字，错误处理和退出机制仍和以前一样工作：

```
Enter text: 50 2500.0 Enter text: 40.5 1640.25 Enter text: 1.23E-100 1.5129e-200 Enter text: hack Bad!Bad!Bad!
```

Bad!Bad!Bad!Bad!Bad! Enter text: stop Bye 注释 (NOT E) : 或者用 eval 和 exec 跑任何东西: Python 内置的 eval 调用——我们在第 5 章和第 9 章用它把字符串和文件里的数据转换出来——在这里也能替代 float 使用, 而且它会接受任意表达式的输入 ("2 100" 会是一个合法而奇特的输入, 尤其当我们假设程序处理的是年龄时!)。这是一个强大的概念, 同时也向之前章节提到的那些安全风险敞开了大门。如果你无法信任代码字符串的来源, 就要使用像 int 和 float 这样更聚焦、更受限的转换工具。Python 的 exec——第 3 章里用它来运行从文件读出的代码——与 eval 类似 (但它假定字符串是一条语句而非表达式, 而且没有结果), 它的 compile 调用则把频繁使用的代码字符串预编译成字节码对象以提升速度。对这些函数中的任意一个运行 help 可以获得更多细节。我们还会在第 25 章用 exec 通过名称字符串导入模块——那是它更动态的用途之一。

深度理解

- 核心概念: 本小节汇聚了两条信息: ①try 可以显著减少错误处理代码量; ②没有 isfloat 方法的事实说明, 用 try/except 拦截 float() 的异常, 是支持所有浮点格式 (包括 1.23E-100 科学计数法、负号、小数点) 最省力的方案。
- 底层实现: float('1.23E-100') 走 C 语言的 strtod 解析路径, 成功则返回双精度浮点数并存入 PyFloat 对象, 失败就抛 ValueError。这正是"异常即控制流"的教科书案例: 解析器本身在"失败"瞬间就给了你分流信号, 无需你手工枚举所有合法格式。
- 设计原因: 为什么没有 isfloat? 因为"浮点格式"的集合太

大且不断演化 (Python 3.11+ 甚至允许下划线分隔 100 0.5) 。与其定义"合法浮点字符串"的准确集合, 不如让 float() 自己当权威裁判——唯一事实源 (single source of truth) 。这也是 EAFP 优于 LBYL 的深层原因。

- 安全警示: NOTE 里的 eval 是重武器。eval("import('os').system('rm -rf /')") 这类字符串一旦插进用户输入, 等于开门揖盗。业界铁律: 绝不 eval 不可信输入; eval/exec 只能用于受信任的固定代码字符串。作者连续两处强调"security issues"就是为此。

- 生活例子: try/except 像"先锁门再说"——你不需要预先判断门锁是否完好 (枚举所有故障) , 只管锁 (float()) , 锁不上 (异常) 再处理。

- 初学者误区:

- 以为 EAFP 就是"不管三七二十一直接跑", 其实它依然要求知道捕获什么异常。

- 用 eval 替代 int/float 做简单输入校验——为了一时方便埋下 RCE (远程代码执行) 级别的隐患。

- 忽略 except: 会连 StopIteration、KeyboardInterrupt 也吞掉的边界问题。

10.9.7 三层嵌套的代码 (Nesting Code Three Levels Deep)

英文原文

Let's look at one last mutation of our code. Nesting can take us even further if we need it to—we could, fo

r example, extend our prior integer-only script to branch to one of a set of alternatives based on the relative magnitude of a valid input:

```
python while True: reply = input('Enter text:') if reply == 'stop': break
```

```
elif not reply.isdigit(): print('Bad! 8) else: num = int(reply) if num < 20: print('low') else: print(num 2) This version adds an if statement nested in the else clause of another if statement, which is in turn nested in the while loop. When code is conditional or repeated like this, we simply indent it further to the right. The net effect is like that of prior versions, but we'll now print "low" for numbers less than 20:
```

```
Enter text: 19 low Enter text: 20 400 Enter text: hack Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad! Enter text: stop Bye
```

中文翻译

让我们再看看这段代码的最后一次变形。如果我们需要，嵌套可以走得更远——例如，我们可以把之前那个“仅限整数”的脚本扩展一下，让它根据合法输入的数值大小，在一组备选分支中选择其一：
python while True: reply = input('Enter text:') if reply == 'stop': break elif not reply.isdigit(): print('Bad! 8) else: num = int(reply) if num < 20: print('low') else: print(num 2) print('Bye') 这个版本在另一条 if 语句的 else 子句里又嵌套了一条 if 语句，而这条外

层的 if 语句又嵌套在 while 循环里。当代码像这样需要条件分支或重复执行时，我们只需要把它往右缩进得更深即可。最终效果与之前的版本类似，只不过现在对于小于 20 的数字，我们会打印 "low"：Enter text: 19 low Enter text: 20 400 Enter text: hack Bad!Bad!Bad!Bad!Bad!Bad!Bad!Bad! Enter text: stop Bye

深度理解

·核心概念：本小节是嵌套的"极限演示"——while 包 if/elif/else，else 里再包一层 if/else，形成三层缩进结构。它直观展示：嵌套没有数量上限，每深一层逻辑，就往右缩进一级。同时暗示了"反锯齿"原则：嵌套层级越多，代码越难读——这正是后续章节会引入 match (3.10+)、提前 continue、函数抽离等扁平化手段的动机。

·底层实现：三层结构在字节码里是三个嵌套的"跳转决策区间"。内层 if num < 20 的跳转目标（打印'low'或求平方）完全落在外层 else 的"受保护区间"之内；缩进层级在编译期就已固化成 INDENT/DEDENT 序列，任何跳转都不会跨出它所属的块。缩进不仅塑造视觉，也塑造字节码跳转的作用域。

·设计原因：作者用这个例子回答"嵌套到底能多深"——想多深就多深，规则唯一且一致。但紧接着的下文（Chapter Summary 之后的实践建议）暗示：深嵌套是代码坏味道，读到第 3 层就应考虑重构。

·生活例子：像俄罗斯套娃——大娃娃（while）套中娃娃（if/elif/else），中娃娃里再套小娃娃（内层 if/else）。

打开的顺序与闭合的顺序严格相反，就像缩进的进入与退出。

·初学者误区：

·逻辑上把"嵌套 if"写成分层的 elif——它们语义不同（嵌套 if 有父子关系，elif 是并列分支），错一层缩进就错一个语义。

·以为"缩进层级越多越专业"——实际上每多一层，IndentationError 概率和阅读成本都指数上升。

·忘记 break 只跳出最内层循环——如果你在嵌套循环里写 break，它只打断最近的 while/for。

代码分析

```
while True:
    reply = input('Enter text:')          # 1
    if reply == 'stop':                    # 2
        break
    elif not reply.isdigit():              # 2
        print('Bad!' * 8)
    else:                                  # 2
        num = int(reply)
        if num < 20:                       # 3
            print('low')
        else:
            print(num ** 2)
print('Bye')                               # 0
```

解释器如何处理：tokenizer 按行计数缩进层级：0 → 4（while 体）→ 8（if/elif/else）→ 12（else 里的内层 if/else）。每一级对应一次 INDENT，每退出一级对应一次 DEDENT，语法分析器据这些记号把 12 空格的 print('lo

w') 正确挂到"外层 else 之下的内层 if 之下"。字节码的跳转目标因此精确绑定: num < 20 为真跳'low', 为假跳到 num 2; 整段逻辑嵌在外层 else 的代码区间内, 最后由循环的尾部指令收束, 随后执行文件级 (0 缩进) 的 print('Bye')。

设计思想: 三层嵌套展示了"缩进模型"的完全一致性——从一层到三层, 规则没有改变, 只是重复运用。但作者也借此提醒: 能用结构压平的逻辑, 就不要让它沉到三层以下——缩进是免费的, 但可读性不是。

10.10 本章小结 (Chapter Summary)

英文原文

That concludes our first look at Python statement syntax. This chapter introduced the general rules for coding statements and blocks of code. As you've learned, in Python we normally code one statement per line and indent all the statements in a nested block the same amount (indentation is part of Python's syntax). However, we also looked at a few exceptions to these rules, including continuation lines and single-line tests and loops. Finally, we put these ideas to work in an interactive script that demonstrated a handful of statements and showed statement syntax in action. In the next chapter, we'll start to dig deeper by going over each of Python's basic procedural statements in de

pth. As you'll see, though, all statements follow the same general rules introduced here.

中文翻译

我们对 Python 语句语法的第一轮介绍到此结束。本章介绍了编写语句和代码块的一般规则。正如你所学到的，在 Python 中我们通常一行一条语句，并让嵌套代码块中的所有语句缩进相同距离（缩进是 Python 语法的一部分）。不过，我们也看了这些规则的几个例外，包括续行、单行测试和单行循环。最后，我们在一个交互式脚本中把所有这些思想付诸实践——它演示了好几条语句，并展示了语句语法在实战中的样子。在下一章，我们将开始更深入的探索，逐一钻研 Python 的每一条基本过程式语句。不过你会看到，所有语句都遵循本章介绍的那套通用规则。

深度理解

·核心概念：本章的"总纲"就是两句话——一行一条语句（换行即终止），嵌套块等距缩进（缩进即块）。加三个特例：分号挤行、括号续行、单行复合语句。再加一个实战演练：交互循环。

·底层实现：把全文串起来看，CPython 的词法/语法规则是：物理行 → 逻辑行（拼接规则）→ 记号流（NEWLINE/INDENT/DEDENT/COLON）→ 语法树 → 字节码。本章讲的就是这条管线的前一半。

·设计原因：作者特意在每章小结里给出"下一章预告"，是为了建立递进结构：第 11 章逐条讲解所有过程式语句（赋值、print、循环等）时，你将反复用上本章的语法骨架

- 生活例子：像先学会"汉语的句子结构"（主谓宾、标点）再学"各种修辞手法"。本章就是标点与句式总规则。
 - 初学者误区：以为"下一章才学语句，本章可以略过"——恰恰相反，没有本章的语法模型，第 11 章里每个语句的格式都会"知其然不知其所以然"。
-

10.11 测测你的知识：小测验 (Test Your Knowledge: Quiz)

英文原文

1. What three things are required in a C-like language but omitted in Python? 2. How is a statement normally terminated in Python? 3. How are the statements in a nested block of code normally associated in Python? 4. How can you make a single statement span multiple lines? 5. How can you code a compound statement on a single line? 6. Is there any valid reason to type a semicolon at the end of a statement in Python? 7. What is a try statement for? 8. What is the most common coding mistake among Python beginners?

中文翻译

1. 类 C 语言中必需、但 Python 中省略的三样东西是什么？2. 在 Python 中，一条语句通常是如何终止的？3. 在 Python 中，嵌套代码块里的语句通常靠什么建立关联？4. 你

怎样让一条语句横跨多行？5. 你怎样把一条复合语句写成单行？6. 在 Python 中，在语句末尾敲一个分号有任何正当理由吗？7. try 语句是干什么用的？8. Python 初学者最常见的编码错误是什么？

10.12 测测你的知识：答案 (Test Your Knowledge: Answers)

英文原文

1. C-like languages require parentheses around the tests in some statements, semicolons at the end of each statement, and braces around a nested block of code. Python requires none of these (but adds a :). 2. The end of a line terminates the statement that appears on that line. Alternatively, if more than one statement appears on the same line, they can be separated with semicolons; similarly, if a statement spans many lines, you must terminate it by closing a bracketed syntactic pair. 3. The statements (code lines) in a nested block are all indented the same number of tabs or spaces. 4. You can make a statement span many lines by enclosing part of it in parentheses, square brackets, or curly braces; the statement ends when Python sees a line that contains the closing part of the pair. 5. The body of a compound statement can be moved to the header line after the colon, but only if the body consists of only noncompound statements. 6. Only wh

en you need to squeeze more than one statement onto a single line of code. Even then, this works only if all the statements are noncompound, and it's discouraged because it can lead to code that is difficult to read. 7. The try statement is used to catch and recover from exceptions (errors) in a Python script. It's often an alternative to manually checking for errors in code. 8. Forgetting to type the colon character at the end of the header line in a compound statement is the most common beginner's mistake. If you're new to Python and haven't made it yet, you probably will soon!

中文翻译

1. 类 C 语言要求：某些语句的测试条件外面要有括号、每条语句末尾要有分号、嵌套代码块周围要有大括号。这三样 Python 都不需要（但多加了一个：）。2. 一行的结束会终止该行上的语句。另一种情况是：如果同一行出现多条语句，可以用分号分隔；同样地，如果一条语句横跨多行，你必须闭合一个括号语法对来终止它。3. 嵌套代码块中的语句（代码行）全部缩进相同数量的 Tab 或空格。4. 你可以把语句的一部分包进圆括号、方括号或花括号来让它横跨多行；当 Python 看到包含该括号对右半部分的那一行时，语句就结束了。5. 复合语句的主体可以移到头部行、冒号之后，但仅当主体只由非复合语句（简单语句）组成时才允许。6. 只有当你需要把一条以上的语句挤到同一行代码上时才有理由。即便如此，这也仅在所有语句都是非复合语句时才能工作，而且不推荐使用，因为它可能导致难以阅读的代码。7. try 语句用于在 Python 脚本中捕获异常（错误）并从

其中恢复。它常常是"在代码里手动检查错误"的替代方案。

8. 忘记在复合语句头部行末尾打冒号是初学者最常见的错误。如果你刚接触 Python 而还没犯过这个错，那你可能很快就犯了！

深度理解（答案背后的原理）

·第 1、2、3 题：三题连起来就是本章语法模型的核心等式——去括号（条件测试的）、去分号（行尾）、去大括号（块边界），换来冒号（头部）与换行（终止）、缩进（成块）。

·第 4 题：考的是"括号对开启的隐式续行"——注意答案强调"直到包含配对右括号的行出现"，因为续行并不是无限延续，右括号才是句号。

·第 5 题：考"单行复合语句"的两个约束：主体必须是非复合语句，且附加部分（如 else）不能跟着挤。

·第 6 题：这是全书最"反直觉"的一题——分号在 Python 里合法但几乎无用，唯一正当用途是同一行分隔多条简单语句；行尾分号是纯噪音。

·第 7 题：try 的核心价值是"捕获+恢复"，它是 EAFP 哲学的落点，与 LBYL 的"先检查"相对。

·第 8 题：作者以二十年教学经验断言：漏冒号是新手第一高频错误，而且"你马上就会犯"——这是全书最有亲和力的一处预言。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 解析/爬虫/数据处理脚本：while 循环配合 if ...: break 守卫是"逐行读取直到 EOF/哨兵值"的标准写法，例如 while (line := f.readline()) != "":。
- 命令行工具的用户输入校验：本章的交互循环就是 input() 与 try/except ValueError 组合的雏形，真实项目（如计算器、配置向导）都沿用这一模式；但生产代码应使用具体的 except ValueError: 而不是裸 except:，并配合 if name == 'main': 组织入口。
- 代码生成器/格式化工具：理解"缩进即块、换行即终止"是编写生成 Python 代码的工具（如代码生成器、ORM 模型生成器）的前提——它们必须正确输出缩进层级与括号续行。
- 多行表达式与长参数列表：真实代码库中大量使用括号续行（PEP 8 的"悬垂缩进"风格），例如函数签名跨行、链式调用断行。这也是 black 等格式化工具的默认行为。
- DSL 与安全边界：eval/exec 的"动态执行"在配置加载、表达式计算（如计算器服务）中仍有用途，但必须严格隔离不可信输入——这是每一份安全审计清单上的重点。

与其他语言 (Java/C++/C#) 的区别

维度	Python	C / C++ / Java / C#
块结构 (block)	缩进 + 冒号 (INDENT/DEDENT token)	{ } 大括号
语句终止	换行 (NEWLINE token)	分号 ;
测试条件括号	可选 (且 Pythonic 风格不用)	必需
续行	括号对 (implicit joining), \ 仅遗留	反斜杠在宏中使用, 语言本身无需
单行多语句	分号 (仅简单语句)	分号是常规
空语句	pass (有真实语义)	空语句 ; / 空块
else 配对	由缩进决定, 所见即所得	就近配对 (dangling else 陷阱)
错误处理	异常是控制流的一等公民 (EAFP)	返回值检查为主 (LBYL 传统)
混合 Tab/空格	编译器报错 (TabError)	编译器通常不检查

设计哲学差异的根源: C 家族的语法目标之一是"编译器友好" (定界符明确、可单行压缩、便于一行式宏); Python 的目标是"人友好" (可读性优先、WYSIWYG)。同一种"块"概念, 两种截然不同的实现——这正是理解两大家族的关键。

Python 发展历史背景

· 1991 年, Guido van Rossum 发布 Python 0.9: 从一开始就采用缩进即块的语法。据说 Guido 在设计时受到 ABC 语言 (荷兰 CWI 的教学生态语言) 的影响——ABC 用缩进表示块结构, 其学生用户反馈"代码整洁得像散文", 这让 Guido 决定坚持缩进方案。

·缩进语法的争议与胜利：80 年代末、90 年代初，许多语言设计师（包括一些后来投奔 Python 的 C 程序员）认为缩进语法"激进"。但 Python 用三十多年的生态扩张证明了它的价值——今天，缩进即块甚至成了 Python 最容易被认出的"名片"。

·语言演进中的特例补充：分号分隔、括号续行、\ 续行这些"逃生通道"从早期版本就存在；else 子句在 for/while/try 中的扩展是后来逐步加入的。本章展示的"通用骨架 + 特例"格局，正是这门语言三十年增量演进的缩影。

·Python 3 的语法革命：2010s 之后新语句（match、type、海象 :=）都遵循同一个"头部行 + 冒号 + 缩进块"模板——这证明 30 年前定下的语法骨架到今天依然自治、可扩展。

高级开发者需要掌握的相关知识

·tokenizer 与 line joining 细节：阅读 CPython 源码中 Tokenizer/tokenizer.c（indent 状态机、toknextc 对 \ 的处理、PyNewline 逻辑），可彻底理解 NEWLINE/INDENT/DEDENT 的生成条件——包括"注释行/空行的缩进被忽略"等细节。

·AST 与编译器后端：用 python -m dis 反汇编本章各代码块，观察 POPJUMPIFFALSE、BINARYPOWER、异常表的区间布局；用 ast.dump(parse(...)) 观察 If、While、Try 节点的 body 列表如何体现缩进层级。

·PEP 8 / PEP 20：缩进的规范性（4 空格、绝不混用 Tab）、续行的悬垂缩进规则、EOL 相关 linter 规则（如 E2

31/E502) , 以及 future 与 TabError 的兼容性策略。

·上下文管理器与 with 语句：第 10 章之后的 with 是"缩进块"哲学在资源管理上的延伸——理解缩进块的语义边界（进入/退出协议）比只记语法更有价值。

·格式化工具链：black（强制格式化）、ruff format、isort、pre-commit——它们把"本章的语法规则"变成 CI 中的强制检查，值得每个团队引入。

·异常哲学落地：掌握 EAFP 与 LBYL 的取舍、except 的精确类型匹配、else 子句在 try/for/while 中的语义，以及 raise ... from 链式异常——这决定了你能写出多"Pythonic"的错误处理。

学习建议 (Learning Advice)

·重要程度：★★★★★（5/5 星）。这是 Python 语法的"地基章"，后面的每一条语句、每一个函数、每一类对象都建立在本章的语法模型之上。理解不到位，后续所有代码都将"知其然不知其所以然"。

·应该掌握到什么程度：

·必须背下：三条默认规则（换行终止、缩进成块、冒号头部）+ 三个特例（分号挤行、括号续行、单行复合语句）+ 一句箴言（"代码看起来是什么样，运行起来就是什么样"）。

·必须练会：在 REPL 里故意制造 SyntaxError: expected ':'、IndentationError、TabError，亲眼观察三种报错的形态；亲手把 C/Java 代码逐行改写成 Python，体会"去

括号、去分号、去大括号，加冒号、换缩进"的翻译过程。

·应当理解：缩进为什么是语法而非风格（tokenizer 的 INDENT/DEDENT）、if/elif/else 与 try/except/else 中 else 归属的判断方法（永远看缩进）、EAFP 与 LBYL 的取舍。

·后续应该学习哪些相关内容：

·立即阅读第 11 章（赋值、表达式语句、print 的细节）——把本章语法骨架应用到每条具体语句上。

·随后读第 12 章（if 语句）与第 13 章（while 与 for 循环、循环 else）——你会发现它们全是"头部行 + 冒号 + 缩进块"这一个模板的实例化。

·掌握 with 语句与上下文管理器（第 34 章）时，回看本章的"块"概念，体会"块不仅是视觉结构，也是运行时协议边界"。

·深入阶段阅读 CPython 源码中 tokenizer 的缩进处理，以及 dis/ast 模块，把本章的语法规则"看到字节码里去"。

·最后，对照 PEP 8、black 格式化文档，养成"代码整洁是语法要求"的职业习惯——这会让你的代码同时赢得人和机器的尊重。